Auditoria Técnica e Funcional Completa

Módulo Agendamento — ClinicaOS / Health Hub Pro (patientpal-secure)

Data	14/07/2026
Branch	main — commit ff7fde35 (repositório atualizado antes da auditoria, sem alterações locais em conflito)
Escopo	Agendamento (Agenda clássica, Agenda V2 piloto, Agenda Express), disponibilidade médica, recepção, check-in,
Método	Leitura de código-fonte, migrations SQL e documentação interna — 6 investigações paralelas cobrindo áreas distin
Referência	Auditoria anterior de 11/07/2026 (relatorio-agendamento-achados.pdf) usada como baseline — cada achado dela f

Resumo executivo

O sistema tem **três implementações independentes** para criar agendamento (Agenda clássica, Wizard da Agenda V2, Agenda Express) e **duas** para reagendar/cancelar (clássica e V2), sem uma camada central que garanta as mesmas regras nos três. A auditoria de 11/07/2026 já havia identificado dois riscos críticos de sobreposição de horário; dois commits de 13/07/2026 corrigiram esses riscos **parcialmente** — apenas no caminho do Wizard novo da Agenda V2. O caminho mais usado no dia a dia (botão "Novo agendamento"/atalho de teclado **N** da Agenda clássica) e a Agenda Express continuam com o mesmo tipo de falha, e o banco de dados não tem nenhuma constraint que impeça dois agendamentos sobrepostos para o mesmo médico — a proteção depende inteiramente do código do cliente respeitar a regra.

Dois achados de segurança se destacam por gravidade: uma função do banco (`buscar_paciente_contato`) está liberada para chamadas **anônimas** (sem login) e devolve nome, CPF, telefone e data de nascimento de pacientes de qualquer clínica; e a checagem de "o horário está dentro do expediente do médico" só roda no servidor se o **próprio cliente** mandar um sinalizador dizendo que quer essa checagem — não há garantia no banco.

Do lado operacional, um agendamento cancelado pode continuar aparecendo como "aguardando" na fila de triagem e pode ser promovido automaticamente para "em atendimento"; o link público de check-in não valida se o agendamento foi cancelado; e cancelar/marcar falta em um agendamento já pago não exige estorno — o dinheiro fica "confirmado" no financeiro mesmo com o atendimento cancelado.

Severidade	Qtd.	Descrição
Crítica	11	Duplicação/sobreposição de agendamento, bypass de validação, vazamento de dados entre clínicas
Alta	14	Inconsistência financeira, dado clínico sem rastreabilidade, falhas de fluxo/status
Média	18	Duplicação de lógica entre v1/v2, lacunas de validação, dívida técnica relevante
Baixa	4	Código morto, defesa em profundidade ausente (mitigada hoje), limitação de modelo
Total	**47**	

Relação de abas, telas e fluxos analisados

Agenda clássica (v1) — src/routes/_authenticated/app.agenda.tsx (6046 linhas) — visualizações "Dia" e "Por médico"; é a agenda usada por padrão. Não foram encontradas visualizações separadas de semana, mês, sala ou unidade — são filtros dentro da view de dia, não abas distintas.

Agenda V2 (piloto) — src/routes/_authenticated/app.agenda-v2.tsx + src/components/agenda-v2/* — restrita a papéis admin/gestor, atrás de feature flag, marcada "piloto" e noindex. Lista de sessões do dia com KPIs, insights de IA, wizard de novo agendamento e modal de reagendamento.

Agenda Express — src/routes/_authenticated/app.agenda.express.tsx (595 linhas) — terceira implementação de criação de agendamento, com modelo de disponibilidade próprio (RPC get_horarios_disponiveis), independente das outras duas.

Disponibilidades — src/routes/_authenticated/app.disponibilidades.tsx — configuração de agendas por médico, disponibilidade semanal, geração de slots; aba de recursos de enfermagem (src/components/enfermagem-horarios-parts.tsx).

Recepção — src/routes/_authenticated/app.recepcao.tsx — fila de recepção do dia.

Check-in (interno) — src/routes/_authenticated/app.checkin.tsx (750 linhas) — confirmação de presença pela equipe.

Check-in (link público) — src/routes/checkin.$token.tsx + RPC checkin_agendamento — paciente confirma presença por link recebido, sem login.

Totem — src/routes/totem.tsx — emissão de senha (fila senhas), não interage com agendamentos/fluxo_etapa.

Autoatendimento — src/routes/autoatendimento.tsx (957 linhas) — kiosk de check-in + agendamento + pagamento.

Triagem — Enfermagem — src/routes/_authenticated/app.triagem-enfermagem.tsx (518 linhas) — fila de triagem, chamada de senha, registro de triagem.

Atendimento (médico/IA) — src/routes/_authenticated/app.atendimento-ia.$agendamentoId.tsx + app.atendimento-ia.index.tsx — tela de atendimento do médico e fila correspondente.

Caixa — src/routes/_authenticated/app.caixa.tsx (3148 linhas) — cobrança, pagamento agrupado, gratuidade, fila do caixa.

Orçamentos vinculados à agenda — src/routes/_authenticated/app.orcamentos-agenda.tsx — shell de vínculo entre orçamento e agendamento.

Estorno financeiro — src/routes/_authenticated/app.financeiro.estorno.tsx + src/lib/estornar-lancamento.ts.

WhatsApp / Nina (bot) — src/routes/api/public/whatsapp.$clinicaId.ts (webhook) + src/lib/whatsapp.server.ts + src/lib/whatsapp.functions.ts.

Não encontrado no código (mencionado no pedido de auditoria, mas sem implementação correspondente): visualização de calendário mensal/semanal dedicada, visualização por sala/consultório ou por unidade como aba separada, lista de espera formal, agendamentos recorrentes, histórico de alterações como tela dedicada (existe log de auditoria em audit_log, não uma tela de histórico do agendamento), bloqueio manual de agenda como funcionalidade de produto (ver achado MED-07).

Relação de arquivos analisados

src/routes/_authenticated/app.agenda.tsx, src/routes/_authenticated/app.agenda-v2.tsx, src/routes/_authenticated/app.agenda.express.tsx, src/routes/_authenticated/app.disponibilidades.tsx, src/routes/_authenticated/app.recepcao.tsx, src/routes/_authenticated/app.checkin.tsx, src/routes/checkin.$token.tsx, src/routes/totem.tsx, src/routes/autoatendimento.tsx, src/routes/_authenticated/app.triagem-enfermagem.tsx, src/routes/_authenticated/app.atendimento-ia.$agendamentoId.tsx, src/routes/_authenticated/app.atendimento-ia.index.tsx, src/routes/_authenticated/app.caixa.tsx, src/routes/_authenticated/app.orcamentos-agenda.tsx, src/routes/_authenticated/app.financeiro.estorno.tsx, src/routes/api/public/whatsapp.$clinicaId.ts, src/components/agenda-v2/novo-agendamento-wizard.tsx, src/components/agenda-v2/reagendar-modal.tsx, src/components/agenda-v2/agenda-v2-shell.tsx, src/components/agenda-v2/session-card.tsx, src/components/agenda-v2/patient-drawer.tsx, src/components/enfermagem-horarios-parts.tsx, src/components/financeiro/lancamento-dialog.tsx, src/components/financeiro/EstornoAtendimentosPanel.tsx, src/components/orcamentos/conversao-orcamento-dialog.tsx, src/lib/agenda/criar-agendamento.functions.ts, src/lib/agenda/reagendar-agendamento.functions.ts, src/lib/agenda/status-agendamento.functions.ts, src/lib/agenda/aviso-limite-pendentes.ts (+ .test.ts), src/lib/agenda/contagem.ts, src/lib/agenda/refs-cache.ts,

src/lib/agenda-v2/session-detect.ts, src/lib/estornar-lancamento.ts, src/lib/cb-regras.ts, src/lib/whatsapp.server.ts, src/lib/whatsapp.functions.ts, src/lib/date-utils.ts, src/hooks/use-realtime-refresh.ts, src/hooks/use-agenda-v2-flag.ts, src/hooks/use-clinica.tsx, src/integrations/supabase/types.ts, docs/agenda/arquitetura.md, docs/regras-de-negocio.md, docs/contexto-do-projeto.md (documentação interna), ~40 migrations em supabase/migrations/ relacionadas a agendamentos, disponibilidade, WhatsApp, senhas, triagem, RLS/permissões e RPCs financeiras (listadas na seção de banco de dados)

Mapa de status do agendamento

Coluna `agendamentos.status` **(enum real do banco — fonte de verdade):** agendado, confirmado, realizado, cancelado, faltou.

Coluna `agendamentos.fluxo_etapa` **(enum separado, controla a posição no fluxo operacional — recepção/triagem/atendimento, independente de status):** aguardando_recepcao, recepcao, caixa, triagem, atendimento, exame, finalizado.

Frontend Agenda clássica (v1): agendado, confirmado, realizado, cancelado, faltou — bate exatamente com o enum do banco. **Frontend Agenda V2:** agendado, confirmado, **em_atendimento**, realizado, cancelado, faltou — "em_atendimento" não existe no banco (ver CRIT-06).

Status "paralelos" encontrados fora da coluna oficial (mesmo domínio modelado de forma isolada, risco de confusão): `fin_lancamentos.status` (pendente/confirmado/cancelado — domínio financeiro, não confundir com status do agendamento); legenda visual da Agenda clássica com 10 rótulos de cor que misturam status, fluxo_etapa e flags de estorno; componentes não utilizados em produção (EstornoAtendimentosPanel.tsx, conversao-orcamento-dialog.tsx) com um terceiro vocabulário próprio de status.

Mapa de transições de status observadas

De → Para	Onde	Validação aplicada
agendado/confirmado → realizado	mudarStatus (v1), atualizarStatusAgendamento (v2)	Checa papel (v2) bloqueia se data futura. NÃO valida que veio de "confirmado" -
qualquer → cancelado	idem	NENHUMA validação de status de origem nem de pagamento confirmado (C
realizado → confirmado (reabertura)	reabrirAtendimento (rota separada, v1)	Papel restrito; trata estorno de repasse; NÃO estorna a receita automaticam
qualquer → qualquer (dropdown linha única)	app.agenda.tsx:5475-5479	NENHUMA restrição de estado de origem (ALTA-02) — ignora a rota de rea
agendado/confirmado → em_atendimento (check-in v2)	checkin.tsx	Falha no banco por enum inexistente (CRIT-06).
reagendamento (troca de horário) v1	RPC reagendar_atendimento	Transacional, FOR UPDATE, revalida destino.
reagendamento (troca de horário) v2	reagendarAgendamento.functions.ts	Bloqueia se realizado/cancelado/faltou, mas 3 updates separados sem trans
cancelamento em cascata de pacote	app.agenda.tsx / status-agendamento.functions.ts	Exclui irmãos já cancelados da lista — correto neste caminho específico.

Transições ausentes/perigosas destacadas: não existe validação de máquina de estados — qualquer status pode virar qualquer outro pelo menu de linha única, inclusive "realizado" → "cancelado" sem passar pelo fluxo de estorno de repasse, e "cancelado" com pagamento confirmado sem exigir estorno primeiro.

Mapa do fluxo completo do agendamento

1. Configuração de disponibilidade do médico (app.disponibilidades.tsx) — regras semanais, vigência, intervalo.
2. Geração de slots — roda 100% no navegador (MED-04), grava linhas "DISPONÍVEL" direto em agendamentos.
3. Criação do agendamento — três caminhos paralelos e não equivalentes: clique em slot (protegido), "Novo agendamento"/atalho N/deep-link WhatsApp (CRIT-01, CRIT-02), Agenda Express (CRIT-03).
4. Confirmação — status muda para "confirmado"; nenhuma notificação automática associada (WhatsApp não dispara confirmação/lembrete automático — ver observação A1 do agente de WhatsApp).
5. Recepção — fila por fluxo_etapa; não filtra status=cancelado de forma consistente (CRIT-09).
6. Check-in — interno (checkin.tsx), por link público sem validação (CRIT-10), ou por totem/autoatendimento (ALTA-13).
7. Triagem — enfermagem chama senha e registra triagem; sem idempotência (ALTA-14); mostra cancelados (CRIT-09).
8. Atendimento médico — tela pode promover agendamento cancelado a "em atendimento" automaticamente (CRIT-09); prontuário sem vínculo rastreável ao agendamento (ALTA-04).

9. Pagamento — caixa cobra por atendimento ou em grupo; pagamento agrupado com falha parcial não reverte avanço de fluxo (ALTA-01); vínculo com orçamento não-atômico (ALTA-12); regra de cartão de benefícios recalculada divergentemente na fila do caixa (ALTA-11).

10. Cancelamento/reagendamento — cancelar não exige estorno mesmo com pagamento confirmado (CRIT-08); reagendar na V2 não é atômico (CRIT-07); v1 e v2 reimplementam a lógica de forma independente (MED-01).

11. Finalização — "concluir manualmente" não valida pagamento; texto da confirmação sugere que libera repasse, mas isso depende de lançamento financeiro existente (achado citado dentro de CRIT-08).

Achados — Severidade Crítica

Achados que podem causar agendamento duplicado, sobreposição de horário, vazamento de dados entre clínicas ou bypass de regra de negócio essencial. Corrigir com prioridade máxima.

CRIT-01 — "Novo agendamento" da Agenda clássica não consome o slot — duplica linha sobre horário DISPONÍVEL

CRÍTICA

Categoria: Integridade de dados / lógica de negócio (double-booking silencioso) | **Aba/fluxo:** Vendas/Agenda clássica — botão "Novo agendamento", atalho de teclado N, deep-link do WhatsApp (?novo=1)

```
src/routes/_authenticated/app.agenda.tsx:2421-2465 (openNew), :2774,2840 (submit/editing_id), :3849 (botão), :3458-3460 (atalho N), :4069 (campo de horário livre), :832-878 (deep-link WhatsApp) — função/trecho: openNew() sempre chama setEditing(null); submit() envia editing_id: editing?.id ?? null
```

Tabela/RPC/trigger: agendamentos — INSERT (ramo else de src/lib/agenda/criar-agendamento.functions.ts:280-291)

Descrição: Existem dois caminhos de criação na Agenda clássica: clicar numa célula DISPONÍVEL da grade (seta editing = o próprio slot, gera UPDATE seguro) e o botão "Novo agendamento"/atalho N/deep-link do WhatsApp (sempre editing = null, abre formulário com data/hora digitada livremente). Se o horário digitado coincidir com um slot DISPONÍVEL já existente, a validação confirma "existe slot livre" e libera salvar — mas como editing_id é null, o código faz um INSERT puro, nunca tocando na linha DISPONÍVEL original.

Evidência:

```
app.agenda.tsx:2840 — editing_id: editing?.id ?? null criar-agendamento.functions.ts:284-290 — supabase.from("agendamentos").insert(payloadNovo) // nunca atualiza o slot encontrado na validação Correção já aplicada só no Wizard V2 (novo-agendamento-wizard.tsx:331-337, comentário do próprio código): "Com editing_id: null aqui, o slot original ficava intocado... e um agendamento novo era inserido por cima, deixando duas linhas no mesmo horário." Commit a6ec804b (13/07) corrigiu isso — só no wizard.
```

Cenário para reproduzir: 1) Gerar disponibilidade do médico X (cria slots DISPONÍVEL). 2) Apertar N (não clicar na célula). 3) Selecionar médico X, digitar manualmente o horário de um slot já existente. 4) Salvar. Resultado: paciente agendado E a célula continua DISPONÍVEL para outro atendente reservar.

Impacto técnico: Linha "fantasma" DISPONÍVEL permanece reservável indefinidamente; duas linhas físicas para o mesmo médico/horário sem qualquer erro do sistema.

Impacto operacional: Overbooking silencioso — dois pacientes podem chegar para o mesmo horário do mesmo médico; retrabalho de recepção; risco de dupla emissão de guia.

Causa provável: Correção do achado original do PDF de 11/07 foi aplicada só no componente novo (Wizard V2, isolado por feature flag), não replicada em app.agenda.tsx, que é a agenda usada por padrão.

Correção recomendada: Em submit() (app.agenda.tsx), quando editing for null, buscar se existe slot DISPONÍVEL/BLOQUEIO do médico cobrindo exatamente [form.inicio, form.fim] e usar esse id como editing_id (mesma lógica já aplicada no wizard). Alternativa: restringir data/hora em openNew/deep-link a um seletor de slots existentes, eliminando o campo livre quando há agenda gerada.

Risco de regressão: Baixo-médio — muda o ramo de decisão de um formulário já existente; testar cuidadosamente os casos onde não existe slot correspondente (deve continuar criando por INSERT).

Testes necessários: Criar via N em horário coincidente com slot DISPONÍVEL (deve consumir, não duplicar); criar via N em horário sem slot correspondente (deve inserir normalmente); repetir via deep-link do WhatsApp.

CRIT-02 — Caminho de INSERT puro sem trava de concorrência nem constraint no banco

CRÍTICA

Categoria: Concorrência / condição de corrida (check-then-act) | **Aba/fluxo:** Agenda clássica — mesmo caminho do CRIT-01 ("Novo agendamento"/N/deep-link)

`src/lib/agenda/criar-agendamento.functions.ts:145-194 (validação via SELECT, sem lock), :271-291 (trava otimista só no ramo UPDATE) — função/trecho: Ramo else do handler de criarAgendamento — INSERT sem WHERE de concorrência`

Tabela/RPC/trigger: agendamentos (schema em supabase/migrations/20260516192658_...sql — sem UNIQUE/EXCLUDE cobrindo overlap de médico/horário)

Descrição: O commit fbf24236 (13/07) adicionou trava otimista apenas no ramo de UPDATE (editing_id presente): grava snapshot de paciente_nome e usa .eq("paciente_nome", snapshot) como condição extra — proteção correta e atômica. O próprio texto do commit reconhece que o ramo de INSERT puro continua sem proteção. Como o CRIT-01 mostra que editing_id: null é, na prática, o caminho do botão principal (não um caso raro de "encaixe"), a exposição é maior do que o commit sugere.

Evidência:

`criar-agendamento.functions.ts:280-291 — insert(payloadNovo) sem condição de concorrência, sem lock. Mensagem do commit fbf24236: "O caminho de INSERT puro (sem slot pre-existente, ex.: encaixe) continua sem trava de banco." Schema (20260516192658...sql:4-21): CREATE TABLE agendamentos sem UNIQUE/EXCLUDE em (medico_id, faixa de tempo).`

Cenário para reproduzir: Dois operadores abrem "Novo agendamento" quase simultaneamente para o mesmo médico e mesmo horário coincidente com um slot DISPONÍVEL. Ambos passam pela validação (só um SELECT, sem lock), ambos fazem INSERT. Sem UNIQUE/EXCLUDE no banco, ambos têm sucesso — dois pacientes no mesmo horário do mesmo médico, sem erro para nenhum dos dois.

Impacto técnico: Condição de corrida clássica sem nenhuma mitigação neste caminho — nem otimista, nem pessimista, nem constraint de banco.

Impacto operacional: Overbooking real e silencioso em uso simultâneo (cenário comum em recepção com múltiplos atendentes); só percebido quando os pacientes chegam.

Causa provável: Correção de concorrência escopada deliberadamente ao caminho "mais comum" (UPDATE de slot pré-existente), sem reavaliar que o botão principal usa INSERT puro.

Correção recomendada: Resolver primeiro o CRIT-01 (estende a trava otimista automaticamente); como defesa em profundidade independente, avaliar constraint de exclusão no banco contra sobreposição por médico (EXCLUDE USING gist, requer btree_gist) — desenho cuidadoso necessário para não quebrar o padrão de slots DISPONÍVEL; apresentar para revisão antes de aplicar.

Risco de regressão: Constraint nova no banco tem risco de quebrar fluxo de geração de slots se mal desenhada — exige validação cuidadosa com a equipe antes de aplicar em produção.

Testes necessários: Teste de carga com 2+ requisições simultâneas de criação para o mesmo médico/horário; confirmar que só uma tem sucesso após a correção.

CRIT-03 — Agenda Express é uma terceira implementação paralela, sem reuso de criarAgendamento, sem trava de concorrência e sem bloqueio de cadastro incompleto

CRÍTICA

Categoria: Múltiplas fontes de verdade / validação ausente / concorrência | **Aba/fluxo:** Agenda Express

```
src/routes/_authenticated/app.agenda.express.tsx:222-248 (confirmarAgendamento), :175-188
(carregarSlots) — função/trecho: confirmarAgendamento() — INSERT direto em agendamentos, sem passar por
criarAgendamento
```

Tabela/RPC/trigger: agendamentos; RPC get_horarios_disponiveis (modelo de disponibilidade próprio, baseado em contagem, diferente do modelo de linhas físicas DISPONÍVEL)

Descrição: Não importa nem usa criarAgendamento. Calcula horários via RPC get_horarios_disponiveis (modelo totalmente diferente do padrão de linhas físicas DISPONÍVEL) e faz INSERT direto e desprotegido no passo de confirmação — sem revalidação, sem trava otimista, sem transação. Não bloqueia paciente com cadastro incompleto (vira só um badge visual, não impede o clique). Não verifica inadimplência de convênio/cartão-benefícios.

Evidência:

```
app.agenda.express.tsx:231-242 — supabase.from("agendamentos").insert({...}) direto, sem nenhuma das
checagens de criarAgendamento. Regra de bloqueio por cadastro incompleto
(criar-agendamento.functions.ts:125-143) inteiramente ausente aqui.
```

Cenário para reproduzir: Concorrência: dois operadores usam a Express simultaneamente para o mesmo médico; ambos recebem o mesmo horário de get_horarios_disponiveis (contagem calculada antes de qualquer INSERT); ambos confirmam; capacidade excedida sem erro. Cadastro incompleto: paciente sem telefone/nascimento, avança e clica "Finalizar" sem completar — agendamento criado mesmo assim.

Impacto técnico: Terceira fonte de verdade com modelo de disponibilidade diferente — risco dos dois modelos conflitarem entre si (não confirmável sem dados ao vivo).

Impacto operacional: Overbooking; dados de contato/nascimento faltantes chegando a fluxos posteriores (NFS-e, cobrança) que dependem desses campos; convênio inadimplente pode ser agendado sem bloqueio via Express.

Causa provável: Agenda Express implementada antes da extração de criarAgendamento (commit anterior em 1 dia às correções) e nunca migrada para reusar a função compartilhada — contraria a regra 2 de frontend.md do próprio projeto.

Correção recomendada: Migrar confirmarAgendamento para chamar criarAgendamento, garantindo as mesmas validações e a mesma trava otimista; reconciliar o modelo de slot virtual da Express com o modelo de linha física DISPONÍVEL.

Risco de regressão: Alto — mudança estrutural na Agenda Express, exige reconciliar dois modelos de dados diferentes; testar extensivamente antes de liberar.

Testes necessários: Criar agendamento via Express com paciente incompleto (deve bloquear); testar concorrência de dois operadores; testar convênio inadimplente.

CRIT-04 — Validação de disponibilidade do médico é decidida pelo cliente, não recalculada no servidor

CRÍTICA

Categoria: Bypass de regra de negócio / confiança indevida no cliente | **Aba/fluxo:** Todos os caminhos de criação (server function compartilhada)

`src/lib/agenda/criar-agendamento.functions.ts:55-59, 152-194 — função/trecho: Tipo CriarAgendamentoInput.checagens.validar_agenda_aberta (boolean vindo do payload do cliente)`

Tabela/RPC/trigger: agendamentos, medico_disponibilidades

Descrição: A única checagem server-side de "horário dentro da disponibilidade do médico" só roda se o payload enviado pelo cliente trouxer checagens.validar_agenda_aberta: true. Esse booleano é decidido no front e chega ao server fn como dado de entrada — nunca recalculado a partir de dados do próprio banco.

Evidência:

`criar-agendamento.functions.ts:152 — if (checagens.validar_agenda_aberta && payload.medico_id) { ... } — nunca recalcula essa decisão, apenas confia no valor recebido.`

Cenário para reproduzir: Um chamador (bug de outro caller futuro, script, ou chamada direta à server function autenticada) envia checagens.validar_agenda_aberta: false — o agendamento é criado em qualquer médico, em qualquer horário, mesmo fora da disponibilidade cadastrada.

Impacto técnico: Bypass total da regra de negócio documentada ("nenhum componente pode decidir client-side", segundo docs/agenda/arquitetura.md) — na prática o servidor confia cegamente no cliente.

Impacto operacional: Agendamento fora do expediente do médico, sobreposição de pacientes, cobrança/repasse indevido, paciente chega e o médico não está disponível.

Causa provável: Extração 1:1 do fluxo client-side original sem repensar o modelo de confiança ao migrar para server function.

Correção recomendada: O servidor deve sempre recalcular se a validação de agenda é necessária (com base em medico_id/enfermagem_recurso_id do próprio payload), nunca aceitar um booleano de confiança vindo do cliente.

Risco de regressão: Médio — mudança de lógica de decisão dentro de uma função já em uso; testar todos os caminhos que hoje dependem do bypass (ex. recursos de enfermagem).

Testes necessários: Chamar a server function diretamente com validar_agenda_aberta: false e confirmar que o servidor ainda valida; testar médico sem disponibilidade cadastrada.

CRIT-05 — RLS de agendamentos não valida disponibilidade nem overlap — só permissão de módulo

CRÍTICA

Categoria: Segurança / integridade de dados (RLS insuficiente) | **Aba/fluxo:** Banco de dados (todas as telas de criação)

`supabase/migrations/20260710135444_harden_module_permissions.sql:150-158 — função/trecho: Políticas agend_select/insert/update/delete`

Tabela/RPC/trigger: agendamentos — RLS

Descrição: As políticas RLS vigentes checam apenas has_module_access(clinica_id,'agenda','write'). Não existe trigger, constraint ou EXCLUDE que impeça INSERT/UPDATE de um agendamento fora de qualquer medico_disponibilidades, sobreposto a outro paciente, ou em horário "bloqueado". O RPC alternativo salvar_agendamento_multi_imagem também faz INSERT/UPDATE direto sem checar disponibilidade.

Evidência:

```
create policy agend_insert on public.agendamentos for insert to authenticated with check (public.has_module_access(clinica_id,'agenda','write'));
```

Cenário para reproduzir: Qualquer usuário com papel que tenha "agenda:write" (recepção, médico, gestor — todos por padrão) pode fazer POST /rest/v1/agendamentos direto (PostgREST/Supabase client), com qualquer medico_id/horário, inclusive fora da disponibilidade real, sem qualquer bloqueio do banco.

Impacto técnico: A "fonte de verdade" documentada ("Nada mais grava em agendamentos. Nada.") é convenção de código, não garantia técnica.

Impacto operacional: Agendamentos fantasmas/incorretos por integração externa, script, ou erro de outro desenvolvedor futuro, sem qualquer alarme.

Causa provável: Modelo de segurança trata RLS como isolamento multi-clínica + permissão de módulo, não como camada de regra de negócio — padrão já conhecido em outros módulos do projeto (docs/regras-de-negocio.md).

Correção recomendada: Mover a validação crítica para dentro de uma função/trigger BEFORE INSERT OR UPDATE no banco, ou uma RPC SECURITY DEFINER como único caminho de escrita com REVOKE INSERT/UPDATE direto na tabela para o papel authenticated.

Risco de regressão: Alto — mudança de modelo de segurança, precisa de planejamento cuidadoso para não quebrar os fluxos legítimos existentes (inclusive o bypass de enfermagem).

Testes necessários: Tentar INSERT direto via API REST fora da disponibilidade após a correção (deve ser bloqueado pelo banco, não só pelo cliente).

CRIT-06 — Status "em_atendimento" usado pela Agenda V2 não existe no enum do banco — check-in quebrado

CRÍTICA

Categoria: Erro de schema / dessincronia frontend-banco | **Aba/fluxo:** Agenda V2 — botão de check-in no card de sessão

`src/lib/agenda/status-agendamento.functions.ts:28-35, src/components/agenda-v2/session-card.tsx:417, src/components/agenda-v2/agenda-v2-shell.tsx:676-684 — função/trecho: STATUS_AGENDAMENTO inclui "em_atendimento"; onChangeStatus(data, "em_atendimento")`

Tabela/RPC/trigger: agendamentos.status — enum public.agendamento_status (supabase/migrations/20260516192658_...sql:2)

Descrição: O enum TypeScript STATUS_AGENDAMENTO (v2) inclui "em_atendimento", e a UI grava esse valor via update direto na coluna status. Mas o enum real do Postgres tem só 5 valores: agendado, confirmado, realizado, cancelado, faltou — confirmado tanto nas migrations quanto no types.ts gerado por introspecção. Não há nenhuma migration com ALTER TYPE adicionando esse valor.

Evidência:

```
CREATE TYPE public.agendamento_status AS ENUM
('agendado','confirmado','realizado','cancelado','faltou'); — sem em_atendimento. session-card.tsx:417 —
onClick={() => onChangeStatus(data, "em_atendimento")} status-agendamento.functions.ts:117-121 —
update(payload as never).in("id", ids) grava esse valor direto na coluna.
```

Cenário para reproduzir: Na Agenda V2, clicar em "Check-in" no card de sessão (mapeado para em_atendimento). A chamada ao Postgres falha com erro de enum inválido, capturado e exibido como toast de erro genérico.

Impacto técnico: Fluxo de check-in da Agenda V2 quebrado; explica por que a v1 usa uma coluna separada (fluxo_etapa) para o mesmo conceito, em vez de sobrecarregar status.

Impacto operacional: Recepção/consultório usando a Agenda V2 não consegue registrar check-in por esse botão.

Causa provável: Dessincronia entre o enum de domínio escrito à mão em TypeScript e o enum real do Postgres — nenhuma migration criada ao introduzir esse valor na v2.

Correção recomendada: Opção (a): migration ALTER TYPE public.agendamento_status ADD VALUE 'em_atendimento' + regenerar types.ts. Opção (b) recomendada: refatorar a V2 para usar a coluna fluxo_etapa já existente, evitando enum paralelo. Mudança de schema — apresentar para revisão antes de db push.

Risco de regressão: Baixo se optar por (b) reusar fluxo_etapa; médio se alterar enum (ALTER TYPE ADD VALUE não é transacional em versões antigas do Postgres — confirmar compatibilidade).

Testes necessários: Clicar em check-in na Agenda V2 após a correção e confirmar sucesso; verificar que a v1 continua funcionando igual.

CRIT-07 — Reagendamento na Agenda V2 não é atômico — regressão em relação à correção aplicada na v1

CRÍTICA

Categoria: Atomicidade / concorrência | **Aba/fluxo:** Agenda V2 — modal de reagendamento

`src/lib/agenda/reagendar-agendamento.functions.ts:167-203 — função/trecho: reagendarAgendamento() — 3 chamadas HTTP separadas, sem transação SQL nem RPC`

Tabela/RPC/trigger: agendamentos (comparar com RPC reagendar_atendimento usada pela v1, migration 20260713190807)

Descrição: A v1 corrigiu esse mesmo problema um dia antes (12→13/07) com uma RPC transacional com FOR UPDATE. Essa correção nunca foi propagada para reagendarAgendamento da V2, que continua fazendo update do slot de destino, tentativa de mover o agendamento, e um finally que tenta reverter — mas isso só protege contra exceção JS dentro do processo, não contra queda de conexão/timeout após o passo já ter sido confirmado no Postgres.

Evidência:

`reagendar-agendamento.functions.ts:170 — update com TMP_MARKER no destino; :186 — update do agendamento; :199 (finally) — tenta reverter, mas não é transação real. Comparar com o comentário da migration 20260713190807: "Move as 3 etapas para uma única transação Postgres: tudo-ou-nada." — aplicado só em v1.`

Cenário para reproduzir: Derrubar a conexão do server function entre o update do destino e o update do agendamento (timeout de rede, reinício do servidor). O slot de destino fica marcado com paciente_nome="DISPONÍVEL_REAGENDADO_TMP" permanentemente (sem job de limpeza), bloqueando aquele horário até correção manual.

Impacto técnico: Ausência de atomicidade; sem FOR UPDATE, duas requisições concorrentes podem escolher o mesmo destSlot e a segunda sobrescreve sem checar se ainda está livre (TOCTOU).

Impacto operacional: Slot fantasma preso no destino; duas recepcionistas usando a V2 podem reagendar pacientes diferentes para o mesmo horário do mesmo médico sem erro.

Causa provável: V2 implementada com estratégia de modelo diferente (swap de slot preservando id) e não herdou a correção de atomicidade aplicada na v1.

Correção recomendada: Criar ou reaproveitar uma RPC transacional para a V2, com FOR UPDATE no slot de destino e revalidação dentro da transação, espelhando reagendar_atendimento adaptada ao modelo "preserva id".

Risco de regressão: Médio — mudança na função de reagendamento da V2, que é piloto (menor exposição de usuários hoje, mas deve ser corrigido antes de promover a V2 a padrão).

Testes necessários: Simular falha de rede entre os passos do reagendamento V2; confirmar que não sobra slot fantasma; testar reagendamento concorrente de dois usuários para o mesmo destino.

CRIT-08 — Cancelamento/falta não bloqueia nem exige estorno de agendamento já pago

CRÍTICA

Categoria: Integridade financeira / regra de negócio ausente | **Aba/fluxo:** Agenda clássica e V2 — menu de status de qualquer agendamento

```
src/routes/_authenticated/app.agenda.tsx:2988-3041 (mudarStatus),
src/lib/agenda/status-agendamento.functions.ts:90-115 — função/trecho: mudarStatus() /
atualizarStatusAgendamento() — nenhuma checagem de pagosSet/lançamento antes de permitir
cancelado/faltou
```

Tabela/RPC/trigger: agendamentos.status, fin_lancamentos (sem trigger de sincronização entre as duas)

Descrição: Nem mudarStatus (v1) nem atualizarStatusAgendamento (v2) verificam se o agendamento tem pagamento confirmado antes de permitir a transição para cancelado/faltou. A única ação vinculada a orcamento_id é desvinculá-lo — não há estorno automático nem bloqueio. Confirmado de forma independente por dois agentes de auditoria (um pelo lado do fluxo/status, outro pelo lado financeiro/caixa). O guard equivalente EXISTE em outra função do mesmo arquivo (remove(), que bloqueia exclusão de horário pago) mas foi omitido especificamente aqui.

Evidência:

```
app.agenda.tsx:2967-2970 (remove(), BLOQUEIA) — if (pagosSet.has(a.id)) { toast.error("Este agendamento
já foi pago. Estorne no Financeiro antes de liberar."); return; } app.agenda.tsx:3013-3040
(mudarStatus(), NÃO checa pagosSet) — só limpa orcamento_id ao cancelar, sem checar pagamento. Não há
trigger de banco sincronizando agendamentos.status com fin_lancamentos (confirmado por busca nas
migrations).
```

Cenário para reproduzir: Abrir um agendamento com pagamento já confirmado; usar o menu "..." → "Cancelado" (ou "Faltou"). A chamada roda sem nenhuma verificação de pagamento.

Impacto técnico: Nenhuma trava impede a transição; a única defesa depende do usuário lembrar de estornar manualmente antes.

Impacto operacional: Reconciliação financeira mostra receita ativa vinculada a atendimento cancelado.

Impacto financeiro/clínico: Risco de duplo faturamento se o paciente for reagendado e cobrado de novo; relatórios de faturamento por período contam receita "fantasma" de agendamentos cancelados; dinheiro fica "confirmado" no caixa para um atendimento que não vai acontecer.

Causa provável: Padrão de proteção (pagosSet.has) implementado em remove() mas não replicado em mudarStatus()/atualizarStatusAgendamento — divergência de implementação dentro do mesmo módulo.

Correção recomendada: Adicionar o mesmo guard de remove() em mudarStatus() (v1) e atualizarStatusAgendamento (v2), bloqueando cancelamento/falta direto quando há pagamento confirmado, e/ou exigindo fluxo de estorno obrigatório antes.

Risco de regressão: Baixo-médio — adiciona uma validação a um fluxo existente; testar que cancelamentos legítimos (sem pagamento) continuam funcionando normalmente.

Testes necessários: Tentar cancelar/marcar falta em agendamento pago (deve bloquear ou exigir estorno); tentar em agendamento não pago (deve funcionar normalmente).

CRIT-09 — Agendamento cancelado continua na fila de triagem e pode ser promovido a "em atendimento" automaticamente

CRÍTICA

Categoria: Dessincronia status × fluxo_etapa (fonte dupla de verdade) | **Aba/fluxo:** Triagem — Enfermagem, Atendimento (médico/IA)

```
src/routes/_authenticated/app.triagem-enfermagem.tsx:122-129,
src/routes/_authenticated/app.atendimento-ia.$agendamentoId.tsx:105-141,
src/lib/agenda/status-agendamento.functions.ts:90-99 — função/trecho: Query de fila de triagem sem
.neq("status","cancelado"); carregarAgendamento() promove fluxo_etapa para "atendimento" automaticamente
se houver pagamento
```

Tabela/RPC/trigger: agendamentos (status, fluxo_etapa)

Descrição: Cancelar um agendamento só altera status (e orcamento_id); nunca toca em fluxo_etapa. A tela de triagem consulta agendamentos filtrando apenas clinica_id, fluxo_etapa='triagem' e a data — sem excluir cancelados. Além disso, a tela de atendimento do médico busca o agendamento só por id (sem checar status) e, se houver pagamento, promove automaticamente fluxo_etapa para "atendimento" — mesmo que status seja "cancelado".

Evidência:

```
app.triagem-enfermagem.tsx:122-129 — .eq("fluxo_etapa","triagem") sem .neq("status","cancelado").
status-agendamento.functions.ts:90-99 — cancelamento nunca toca fluxo_etapa.
atendimento-ia.$agendamentoId.tsx:135-140 — if (status.pago && ag.fluxo_etapa !== "atendimento") {
update({fluxo_etapa:"atendimento"}) } sem checar status do agendamento.
```

Cenário para reproduzir: Paciente faz check-in (fluxo_etapa='triagem'); recepção cancela o agendamento na Agenda. O card do paciente continua aparecendo normalmente na fila de Triagem, como se estivesse aguardando atendimento; se pago, a tela de atendimento promove sozinha para "em atendimento" ao ser aberta.

Impacto técnico: Leitura sem filtro de status; nenhuma trigger/constraint garante consistência status ↔ fluxo_etapa; a fila do médico (atendimento-ia.index.tsx) tem o mesmo problema.

Impacto operacional: Enfermagem pode chamar e atender paciente com consulta cancelada; médico pode abrir e "atender" (e salvar prontuário) um agendamento cancelado, distorcendo agenda/financeiro/repasse.

Causa provável: Função de cancelamento foi projetada para "só alterar o status persistido" (SRP, conforme comentário do próprio arquivo) e nenhuma tela de fila foi atualizada para compensar com filtro de status.

Correção recomendada: Adicionar .neq("status","cancelado") em todas as queries de fila (triagem, atendimento); e/ou fazer atualizarStatusAgendamento também setar fluxo_etapa para um estado terminal quando novo_status='cancelado'.

Risco de regressão: Baixo — adição de filtro em queries de leitura; baixo risco de regressão.

Testes necessários: Cancelar agendamento com check-in feito e confirmar que some da fila de triagem; cancelar agendamento pago e confirmar que a tela de atendimento não promove mais automaticamente.

CRIT-10 — Check-in público via token não valida status/data e é reexecutável indefinidamente

CRÍTICA

Categoria: Regra de negócio ausente em função pública | **Aba/fluxo:** Check-in por link público (/checkin/{token})

```
supabase/migrations/20260520183857_bd17d167-0007-4449-ad8e-3f108d7a0ec5.sql (função checkin_agendamento), src/routes/checkin.$token.tsx:21 — função/trecho: checkin_agendamento(_token) — SECURITY DEFINER, GRANT TO anon, authenticated
```

Tabela/RPC/trigger: agendamentos (fluxo_etapa)

Descrição: A RPC localiza o agendamento só por token_publico e faz UPDATE fluxo_etapa='recepcao' sem checar status <> 'cancelado', sem checar se inicio é hoje, e sem marcar o token como consumido/expirado.

Evidência:

```
SELECT ... status, fluxo_etapa INTO v_ag FROM agendamentos WHERE token_publico = _token LIMIT 1; UPDATE agendamentos SET fluxo_etapa='recepcao', fluxo_atualizado_em=now() WHERE id = v_ag.id; — sem IF v_ag.status='cancelado' THEN ...
```

Cenário para reproduzir: Paciente recebe link /checkin/{token} por SMS/e-mail; a consulta é cancelada depois; o link ainda funciona e reabre fluxo_etapa='recepcao'. O link também pode ser reaberto em qualquer dia (não valida se é a data da consulta) e quantas vezes quiser.

Impacto técnico: Função SECURITY DEFINER sem validação de regra de negócio central, acessível anonimamente.

Impacto operacional: Paciente (ou qualquer pessoa com o link) pode reativar o fluxo de um agendamento cancelado, ou fazer check-in de um dia diferente do agendado, colocando-o na fila da recepção sem que a equipe perceba a inconsistência.

Causa provável: Função implementada focada só no caminho feliz (token válido = permitir), sem os mesmos guards das telas internas de check-in.

Correção recomendada: Dentro da função: IF v_ag.status = 'cancelado' THEN RETURN erro; validar v_ag.inicio::date = current_date (ou janela configurável); considerar token de uso único/expiração.

Risco de regressão: Baixo — validação adicional dentro de uma função já existente; testar que check-ins legítimos continuam funcionando.

Testes necessários: Cancelar agendamento e tentar reabrir pelo link (deve bloquear); tentar check-in em dia diferente do agendado (deve bloquear ou avisar); tentar reexecutar o mesmo link duas vezes.

CRIT-11 — RPC buscar_paciente_contato exposta a chamadas anônimas — vaza PII de pacientes de qualquer clínica

CRÍTICA

Categoria: Segurança / LGPD / RLS via SECURITY DEFINER | **Aba/fluxo:** Backend — usada pela Nina (WhatsApp) para identificar paciente

```
supabase/migrations/20260704155805_c973e66c-24f1-4d31-88c0-8d0394df9918.sql:9-79 — função/trecho: buscar_paciente_contato(_clinica_id, _cpf, _telefone, _nome) — SECURITY DEFINER, GRANT ... TO authenticated, anon, service_role
```

Tabela/RPC/trigger: pacientes (leitura), contratos_assinatura/cb_convenios (leitura)

Descrição: Função SECURITY DEFINER com SET search_path = public (correto), mas: (1) recebe _clinica_id como parâmetro e nunca valida is_member/can_manage_clinica; (2) é concedida a anon — qualquer chamador não autenticado com a anon key pública do projeto pode invocá-la via PostgREST; (3) retorna nome completo, CPF, telefone, data de nascimento e vínculo de convênio/contrato, buscável por fragmento de nome, sufixo de telefone ou CPF.

Evidência:

```
RETURNS TABLE(id, nome, cpf, telefone, data_nascimento, associado, convenio_id, convenio_nome, contrato_id) LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public GRANT EXECUTE ON FUNCTION public.buscar_paciente_contato(uuid,text,text,text) TO authenticated, anon, service_role; Busca por LIKE v_nome_norm || '%' — permite varredura alfabética de pacientes.
```

Cenário para reproduzir: POST https://.supabase.co/rest/v1/rpc/buscar_paciente_contato com apikey: e {"_clinica_id":"","_nome":"MARIA"} — retorna lista de pacientes "MARIA*" da clínica, com CPF/telefone/data de nascimento, sem login.

Impacto técnico: Enumeração de PII em massa; clinica_id não é secreto (aparece na própria URL pública do webhook do WhatsApp).

Impacto operacional: Vazamento de dados pessoais de saúde/identificação (LGPD) de pacientes de qualquer clínica do sistema para qualquer parte não autenticada.

Causa provável: Função criada para ser chamada via supabaseAdmin (service_role) a partir do webhook server-side, mas o GRANT foi concedido de forma ampla demais sem necessidade.

Correção recomendada: REVOKE EXECUTE ON FUNCTION public.buscar_paciente_contato(uuid,text,text,text) FROM anon, PUBLIC; (manter apenas service_role, já que o único chamador legítimo é o webhook via supabaseAdmin); se algum uso autenticado for necessário, adicionar checagem de is_member dentro da função.

Risco de regressão: Baixo — REVOKE não deveria quebrar nenhum fluxo legítimo (o webhook usa service_role, que não é afetado); confirmar que não há nenhum outro caller usando anon/authenticated antes de revogar.

Testes necessários: Confirmar que a chamada anônima passa a retornar erro de permissão após o REVOKE; confirmar que o fluxo da Nina continua funcionando (usa service_role).

Achados — Severidade Alta

Achados com risco de inconsistência financeira, perda de rastreabilidade clínica ou falhas de fluxo que afetam a operação diária.

ALTA-01 — Pagamento agrupado: falha no rateio não bloqueia avanço de fluxo nem impressão de guia

ALTA

Categoria: Atomicidade / integridade financeira | **Aba/fluxo:** Caixa/Agenda — pagamento de múltiplos atendimentos de uma vez

```
src/routes/_authenticated/app.agenda.tsx:4405-4470 — função/trecho: onSavedWithData() — chama fn_registrar_lancamento_e_caixa (atômica) e depois finalizar_pagamento_agrupado (atômica em si, mas não com o passo anterior)
```

Tabela/RPC/trigger: fin_lancamentos, caixa_movimentos, RPC finalizar_pagamento_agrupado (migration 20260713193245)

Descrição: O lançamento "principal" com valor total é criado atomicamente; o rateio entre os N atendimentos do grupo é uma segunda chamada RPC separada. Se o rateio falhar, o código não interrompe: marca todos os IDs como pagos localmente, avança fluxo para todos, e imprime guia agrupada — mesmo sem lançamento real para 2 dos atendimentos.

Evidência:

```
app.agenda.tsx:4412-4414 — if (errRateio) { mostrarErro(...) } — sem return/throw, execução continua; :4416-4421 setPagosSet marca todos como pagos mesmo assim.
```

Cenário para reproduzir: Cobrar 3 atendimentos agrupados; a RPC de rateio falha (rede, lock, constraint). Lançamento principal fica com valor total (não rateado); 2 atendimentos não têm lançamento algum, mas são marcados pagos, avançam de fluxo e têm guia impressa.

Impacto técnico: Estado inconsistente entre agendamentos marcados pagos e fin_lancamentos real.

Impacto operacional: Repasse médico desses 2 atendimentos nunca aparece na tela de repasse (que lista por fin_lancamentos) — fica pendente indefinidamente sem sinalização.

Impacto financeiro/clínico: 2 atendimentos tratados como pagos sem nenhum registro financeiro real.

Causa provável: Duas RPCs atômicas em si, mas não unificadas numa transação só cobrindo criação do principal + rateio.

Correção recomendada: Unificar as duas RPCs numa só chamada, ou tratar erro do rateio como falha bloqueante (não avançar fluxo/imprimir/marcar pago).

Risco de regressão: Médio — mudança no fluxo de pagamento agrupado, testar cuidadosamente cenários de sucesso e falha.

Testes necessários: Simular falha da RPC de rateio e confirmar que nenhum atendimento é marcado como pago incorretamente.

ALTA-02 — Dropdown de status permite qualquer transição sem validar estado atual

ALTA

Categoria: Máquina de estados ausente | **Aba/fluxo:** Agenda clássica e V2 — menu de contexto de cada agendamento

`src/routes/_authenticated/app.agenda.tsx:5475-5479, src/components/agenda-v2/session-card.tsx:405-437 — função/trecho: Menu renderiza Object.keys(STATUS_LABEL) sem desabilitar opções com base no status atual`

Tabela/RPC/trigger: agendamentos.status

Descrição: Nenhuma validação impede cancelar um agendamento já "realizado", ou voltar de "cancelado" direto para "confirmado"/"agendado" sem passar pelo fluxo dedicado reabrirAtendimento (que trata estorno de repasse pago).

Evidência:

```
app.agenda.tsx:5475-5476 — (Object.keys(STATUS_LABEL) as Status[]).map(s => mudarStatus(a, s)}>.
Comparar com reabrirAtendimento (linhas 2192-2249), que tem validação de papel e trata repasse pago — ignorado pelo dropdown genérico.
```

Cenário para reproduzir: Em um agendamento "realizado" com repasse já pago, usar o menu "..." e clicar direto em "Cancelado" — roda sem nenhuma das validações que reabrirAtendimento aplicaria.

Impacto técnico: Bypass do único fluxo que trata estorno de repasse médico.

Impacto operacional: Permite reverter "Faltou"/"Cancelado" para "Confirmado" sem re-liberar corretamente o slot.

Causa provável: Menu genérico construído por iteração sobre todos os status possíveis, sem máquina de estados explícita.

Correção recomendada: Definir tabela de transições válidas por status atual e desabilitar/ocultar opções inválidas; redirecionar "realizado" → outro status para o fluxo reabrirAtendimento já existente.

Risco de regressão: Baixo-médio — regra de UI, testar que todos os fluxos legítimos de mudança de status continuam acessíveis.

Testes necessários: Confirmar que transições inválidas ficam desabilitadas/ocultas; confirmar que "realizado"→outro sempre passa por reabrirAtendimento.

ALTA-03 — Agendamentos de recursos de enfermagem pulam validação de slot E trava de concorrência

ALTA

Categoria: Validação ausente / concorrência | **Aba/fluxo:** Agenda — agendamentos vinculados a enfermagem_recurso_id

```
src/routes/_authenticated/app.agenda.tsx:2816,2844;
src/lib/agenda/criar-agendamento.functions.ts:152,271-279 — função/trecho: validar_agenda_aberta
condicionado a !recursoIds.has(form.medico_id); trava otimista condicionada ao mesmo bloco
```

Tabela/RPC/trigger: agendamentos

Descrição: Para agendamentos de recurso de enfermagem, validar_agenda_aberta é sempre false, então slotPacienteNomeNaValidacao nunca é populado — e a trava otimista (.eq("paciente_nome", snapshot)) nunca é aplicada, mesmo no ramo UPDATE (o caminho "protegido" para todo o resto do sistema).

Evidência:

```
app.agenda.tsx:2844 — validar_agenda_aberta: !!form.medico_id && mudouHorarioOuMedico &&
!recursoIds.has(form.medico_id). criar-agendamento.functions.ts:271-276 — guarda de concorrência só
aplicada se slotPacienteNomeNaValidacao !== null, que para recurso é sempre null.
```

Cenário para reproduzir: Dois operadores atribuem simultaneamente pacientes diferentes ao mesmo horário do mesmo recurso de enfermagem (ex.: sala de curativos) — o segundo UPDATE sobrescreve o paciente do primeiro sem erro para nenhum dos dois.

Impacto técnico: Mesma classe de overbooking silencioso do CRIT-02, restrita a agendamentos de enfermagem/recursos.

Impacto operacional: Paciente perde a vaga sem aviso; recurso pode ficar com dois pacientes conflitantes.

Causa provável: Bypass de checagem de agenda para recursos é intencional (documentado no cabeçalho do arquivo), mas remove também a trava de concorrência introduzida depois, sem revisão cruzada.

Correção recomendada: Aplicar a trava otimista independentemente de validar_agenda_aberta, guardando o snapshot de paciente_nome sempre que editing_id estiver presente.

Risco de regressão: Baixo — extensão de uma proteção já existente para mais um caso.

Testes necessários: Dois operadores atribuindo o mesmo recurso/horário simultaneamente — segundo deve falhar com erro de conflito, não sobrescrever silenciosamente.

ALTA-04 — prontuarios e fin_atendimentos sem vínculo (agendamento_id) com o agendamento de origem

ALTA

Categoria: Rastreabilidade / auditoria clínica | **Aba/fluxo:** Atendimento médico

`src/routes/_authenticated/app.atendimento-ia.$agendamentoId.tsx:296-308,341-354 — função/trecho: INSERT em prontuarios sem coluna agendamento_id (não existe no schema)`

Tabela/RPC/trigger: prontuarios, fin_atendimentos (confirmado sem agendamento_id em src/integrations/supabase/types.ts)

Descrição: A tabela prontuarios não possui coluna agendamento_id. O mesmo vale para fin_atendimentos (vínculo só indireto via lancamento_id, quando existe). A tela de triagem já assume que um paciente pode ter múltiplos agendamentos no mesmo dia com o mesmo médico — nesse caso, prontuários do mesmo dia ficam indistinguíveis por agendamento.

Evidência:

`INSERT INTO prontuarios (clinica_id, paciente_id, medico_id, data, ...) — sem agendamento_id. Schema confirmado em types.ts (Row de "prontuarios" não tem agendamento_id).`

Cenário para reproduzir: Paciente com dois agendamentos no mesmo dia com o mesmo médico (cenário já previsto pela tela de triagem) — os prontuários gerados ficam sem forma determinística de amarrar cada um ao agendamento que o originou.

Impacto técnico: Impossibilidade de rastrear/auditar de forma determinística "este prontuário pertence a este agendamento" no banco.

Impacto operacional: Em caso de disputa/auditoria (glosa de convênio, revisão de prontuário), não há como provar qual consulta agendada gerou qual prontuário quando há mais de um atendimento no mesmo dia.

Causa provável: Modelo de dados criado antes do conceito de fluxo_etapa/multi-agendamento-por-dia, nunca migrado para incluir o vínculo.

Correção recomendada: Adicionar coluna agendamento_id uuid REFERENCES agendamentos(id) (nullable, retrocompatível) em prontuarios e fin_atendimentos; passar agendamento_id no insert desta tela.

Risco de regressão: Baixo — coluna nova nullable, não quebra dado existente.

Testes necessários: Criar prontuário a partir de um agendamento e confirmar que o vínculo é gravado; confirmar que dois atendimentos no mesmo dia geram prontuários distinguíveis.

ALTA-05 — "Confirmar presença" no check-in é escrita incondicional sem lock nem realtime

ALTA

Categoria: Concorrência / lost update | **Aba/fluxo:** Check-in (interno)

```
src/routes/_authenticated/app.checkin.tsx:614-671 (update em 651-657) — função/trecho:
confirmarCheckin() — UPDATE sem .eq("fluxo_etapa", valor esperado)
```

Tabela/RPC/trigger: agendamentos.fluxo_etapa

Descrição: UPDATE incondicional para fluxo_etapa='triagem', sem condicionar ao valor atual. A página não tem subscrição realtime nem polling (diferente de outras telas do mesmo fluxo, que assinam postgres_changes) — a lista só recarrega quando o usuário refaz a busca.

Evidência:

```
app.checkin.tsx:651-657 — update({fluxo_etapa:"triagem",...}).eq("id", item.id) — sem condição de
estado.
```

Cenário para reproduzir: Recepcionista A abre a tela; enquanto isso o paciente já avança para "atendimento" por outra tela/usuário. A lista de A não recarrega automaticamente; se A clicar "Confirmar presença", o agendamento é regredido de volta para "triagem".

Impacto técnico: Ausência de otimistic locking e de realtime nesta tela específica.

Impacto operacional: Paciente já atendido pode reaparecer na fila de triagem/checkin, gerando confusão operacional e possível reatendimento.

Causa provável: Tela criada sem realtime (decisão de escopo aparente — as demais telas do fluxo têm) e update sem guarda de estado.

Correção recomendada: Condicionar o UPDATE a .eq("fluxo_etapa", item.fluxo_etapa) e tratar "0 rows affected" como "estado mudou, recarregue"; adicionar subscrição realtime igual às demais telas.

Risco de regressão: Baixo — adiciona proteção a uma tela existente.

Testes necessários: Simular dois usuários operando o mesmo agendamento simultaneamente; confirmar que o segundo update é rejeitado/avisa quando o estado já mudou.

ALTA-06 — Webhook do WhatsApp sem idempotência real — reentrega duplicada gera resposta e custo duplicados

ALTA

Categoria: Concorrência / confiabilidade de integração | **Aba/fluxo:** WhatsApp / Nina

`src/routes/api/public/whatsapp.$clinicaId.ts:71-117 — função/trecho: Handler POST — não checa se wa_message_id já foi processado antes de responder`

Tabela/RPC/trigger: whatsapp_mensagens (UNIQUE parcial em wa_message_id, mas usada só para o histórico, não como trava de processamento)

Descrição: Existe índice único parcial em wa_message_id, mas o webhook não checa o retorno de erro do insert da mensagem recebida, e mesmo que esse insert falhe por duplicidade, o fluxo segue incondicionalmente para gerar e enviar nova resposta da Nina.

Evidência:

```
await supabaseAdmin.from("whatsapp_mensagens").insert({...}) // erro não verificado, seguido de geração de resposta e envio sem checar duplicidade.
```

Cenário para reproduzir: A Meta Cloud API reenvia o mesmo evento quando não recebe 200 dentro do timeout (comportamento documentado at-least-once) — comum em picos de latência (chamada de IA lenta). Cada entrega duplicada dispara nova chamada de IA e novo envio para o mesmo paciente.

Impacto técnico: Custo duplicado de IA e de mensageria, sem controle de rate/dedupe.

Impacto operacional: Paciente recebe a mesma resposta da Nina 2+ vezes.

Impacto financeiro/clínico: Custo financeiro não controlado em picos (mensagens HSM fora de sessão são cobradas pela Meta).

Causa provável: Índice único existe só para evitar duplicar o registro histórico, não foi promovido a trava de idempotência de processamento (padrão já usado em emitir_senha/chamar_proxima_senha via advisory lock, mas não replicado aqui).

Correção recomendada: Antes do bloco de autorresposta, checar por wa_message_id (ou usar o erro 23505 do insert) e, se já existir, retornar 200 sem reprocessar.

Risco de regressão: Baixo — adiciona uma checagem antes do processamento.

Testes necessários: Reenviar o mesmo payload de webhook duas vezes e confirmar que a segunda não gera nova resposta/envio.

ALTA-07 — Tabela agendamentos sem CHECK constraint fim > inicio

ALTA

Categoria: Integridade de dados | **Aba/fluxo:** Todos os caminhos de criação/edição

```
supabase/migrations/20260516192658_16fee99d-5a9e-442a-89dd-42c2bfee78ef.sql:4-21 (schema sem CHECK);
src/lib/agenda/criar-agendamento.functions.ts (handler sem validação equivalente) — função/trecho: —
```

Tabela/RPC/trigger: agendamentos

Descrição: Diferente de medico_disponibilidades (que tem CHECK hora_fim > hora_inicio), a tabela agendamentos nunca recebeu CHECK equivalente. A validação new Date(form.fim) <= new Date(form.inicio) existe só no cliente (app.agenda.tsx); criarAgendamento não repete essa checagem no servidor (só reagendarAgendamento.functions.ts valida).

Evidência:

```
Busca por CHECK relacionado a fim/inicio em agendamentos nas ~19 migrations que alteram a tabela: nenhum resultado.
```

Cenário para reproduzir: Chamar criarAgendamento diretamente (bypassando a UI) com payload.fim < payload.inicio — o INSERT é aceito.

Impacto técnico: Linhas com intervalo inválido corrompem cálculos de agenda, ocupação, KPIs, e podem quebrar a lógica de "slot cobre o intervalo" usada nas validações de conflito.

Impacto operacional: Dado corrompido silenciosamente aceito pelo banco.

Causa provável: CHECK constraint nunca adicionada nas ~19 migrations que tocam a tabela.

Correção recomendada: ALTER TABLE agendamentos ADD CONSTRAINT agendamentos_fim_gt_inicio CHECK (fim > inicio); + validação espelhada no server function.

Risco de regressão: Baixo-médio — confirmar antes que não existem dados históricos que violem essa regra (poderia impedir a migration de aplicar).

Testes necessários: Tentar criar agendamento com fim <= inicio via chamada direta à server function (deve ser rejeitado após a correção).

ALTA-08 — RPC paciente_cartao_inadimplente acessível por qualquer usuário autenticado de qualquer clínica

ALTA

Categoria: Segurança / RLS via SECURITY DEFINER | **Aba/fluxo:** Backend — checagem de inadimplência na criação de agendamento

```
supabase/migrations/20260611195246_f6141dbc-1432-4a7b-8e69-6aa4ee4640bf.sql:2-51 (+ revoke parcial em 20260611195314) — função/trecho: paciente_cartao_inadimplente(_paciente_id, _clinica_id) — SECURITY DEFINER, sem checar is_member
```

Tabela/RPC/trigger: contratos_assinatura, contrato_mensalidades (via SECURITY DEFINER, contorna RLS dessas tabelas)

Descrição: A função foi criada com GRANT a authenticated e anon; uma migration seguinte revogou anon, mas authenticated de qualquer clínica ainda pode chamar — a função só filtra por _clinica_id recebido como parâmetro, sem validar que o chamador é membro dela.

Evidência:

```
Função usada em criar-agendamento.functions.ts:198-201 via supabase.rpc(...) com o cliente ligado ao JWT do usuário — nesse caminho o RLS normalmente barraria leitura direta, mas a função SECURITY DEFINER contorna isso.
```

Cenário para reproduzir: Usuário autenticado da Clínica A chama supabase.rpc("paciente_cartao_inadimplente", {_paciente_id: , _clinica_id: }) direto do console do navegador — recebe total em aberto, parcelas, convênio e vencimentos de paciente de outra clínica.

Impacto técnico: Contorno de RLS via função definer sem validação de posse.

Impacto operacional: Vazamento de dado financeiro/contratual entre clínicas distintas do mesmo sistema multi-tenant.

Causa provável: Função criada para uso interno de um fluxo já autorizado (dentro de criarAgendamento), mas o RPC em si não replica essa validação e fica exposto a chamada direta.

Correção recomendada: Adicionar checagem de is_member(auth.uid(), _clinica_id) no início da função, com RAISE EXCEPTION se falhar.

Risco de regressão: Baixo — adiciona validação a uma função já em uso interno; confirmar que o fluxo legítimo (dentro de criarAgendamento) continua funcionando.

Testes necessários: Chamar a RPC com paciente/clínica da qual o usuário não é membro (deve ser bloqueado após a correção).

ALTA-09 — CASCADE em anamnese_respostas.agendamento_id apaga dado clínico ao excluir agendamento, sem auditoria

ALTA

Categoria: Governança de dados / cascata de exclusão | **Aba/fluxo:** Agenda — exclusão de agendamento (admin/gestor)

```
supabase/migrations/20260517005734_ffcdf4b7-2974-4ecd-97f3-4d08c4d74825.sql:10-11 — função/trecho: ALTER TABLE anamnese_respostas ADD COLUMN agendamento_id uuid REFERENCES agendamentos(id) ON DELETE CASCADE
```

Tabela/RPC/trigger: anamnese_respostas

Descrição: Diferente de todas as demais FKs apontando para agendamentos(id) (fin_lancamentos, nfse, orcamento_itens usam SET NULL), a resposta de anamnese do paciente é apagada em cascata se o agendamento vinculado for excluído.

Evidência:

```
REFERENCES public.agendamentos(id) ON DELETE CASCADE — único caso de dado clínico (não apenas vínculo) com CASCADE nessa direção.
```

Cenário para reproduzir: Um admin/gestor (único papel autorizado a excluir agendamento) exclui um agendamento antigo pela Agenda; todas as respostas de anamnese associadas àquele agendamento_id são apagadas junto, sem aviso específico sobre esse efeito colateral.

Impacto técnico: Perda de dado sem possibilidade de recuperação via a própria aplicação.

Impacto operacional: Perda de dado clínico (prontuário/anamnese) do paciente — relevante para histórico médico; a exclusão em cascata não passa pelo trigger de auditoria (aplicado em agendamentos, mas não em anamnese_respostas), então a perda não fica registrada.

Causa provável: Modelagem tratou anamnese_respostas como "descartável junto com o agendamento", sem considerar retenção de prontuário.

Correção recomendada: Trocar para ON DELETE SET NULL (preservando a resposta como histórico órfão) ou impedir exclusão de agendamentos com anamneses respondidas vinculadas — decisão de produto, sinalizar para validação humana (regra de dados imutáveis do projeto).

Risco de regressão: Médio — mudança de FK em produção com dado existente; migration precisa considerar registros já órfãos.

Testes necessários: Excluir agendamento com anamnese vinculada após a correção e confirmar que a resposta é preservada (com agendamento_id nulo) em vez de apagada.

ALTA-10 — Checagem "já pago" duplicada e inconsistente em 3 lugares — quebra re-cobrança pós-estorno

ALTA

Categoria: Duplicação de lógica / regra de negócio inconsistente | **Aba/fluxo:** Caixa

```
src/routes/_authenticated/app.caixa.tsx:773-780 (cobrar),
supabase/migrations/20260704171043_82022cf7-28fc-453c-89f7-98f3dac60bc8.sql:123-124 (fila_caixa_hoje) —
ambos sem filtro; comparar com lancamento-dialog.tsx:361-368 (correto) — função/trecho: Checagem de "já
pago" sem .neq("status","cancelado") em 2 dos 3 pontos
```

Tabela/RPC/trigger: fin_lancamentos

Descrição: Como estornarLancamentoReceita só muda status para "cancelado" (não deleta), esse lançamento cancelado continua existindo e continua batendo nas checagens sem filtro. Resultado: depois de um estorno legítimo, o agendamento reaparece na fila do Caixa mas é exibido com badge "PAGO" permanentemente, e o botão "Cobrar" nunca aparece — bloqueando reemissão de cobrança pela tela de Caixa.

Evidência:

```
app.caixa.tsx:773-780 — select fin_lancamentos ... eq("agendamento_id",...) sem
.neq("status","cancelado"). fila_caixa_hoje (SQL) — mesmo padrão sem filtro.
```

Cenário para reproduzir: Estornar um pagamento legítimo; o agendamento reabre fluxo_etapa mas o Caixa continua mostrando "PAGO" e não oferece "Cobrar" de novo.

Impacto técnico: 3 implementações divergentes da mesma checagem de negócio.

Impacto operacional: Recepção/caixa fica impedida de recobrar um atendimento legitimamente estornado, sem alternativa clara na tela.

Impacto financeiro/clínico: Risco também na direção oposta: o caminho de gratuidade (app.agenda.tsx:3250) não faz nenhuma checagem de "já pago" — vulnerável a duplo clique/corrida, sem constraint única no banco.

Causa provável: Lógica de "já pago" implementada 3 vezes de forma independente, sem função/RPC compartilhada.

Correção recomendada: Extrair a checagem para uma única função/RPC reutilizada nos 3 pontos, sempre excluindo status='cancelado'; considerar constraint parcial única (agendamento_id) WHERE tipo='receita' AND status='confirmado' em fin_lancamentos como rede de segurança.

Risco de regressão: Baixo-médio — centralizar lógica existente; testar os 3 pontos de uso após a mudança.

Testes necessários: Estornar um pagamento e confirmar que o Caixa volta a oferecer "Cobrar"; testar duplo clique no fluxo de gratuidade.

ALTA-11 — Fila do Caixa recalcula regra de cartão de benefícios em SQL separado, ignorando carência e limite de uso

ALTA

Categoria: Duplicação de lógica de cálculo / regra de negócio divergente | **Aba/fluxo:** Caixa — fila do dia

`supabase/migrations/20260704171043_82022cf7-28fc-453c-89f7-98f3dac60bc8.sql:71-118 (fila_caixa_hoje) vs. src/lib/cb-regras.ts:37-98 — função/trecho: fila_caixa_hoje reimplementa findRegra/computeValor só com modo/valor/percentual/prioridade`

Tabela/RPC/trigger: cb_convenio_regras

Descrição: src/lib/cb-regras.ts (usado por Agenda e Procedimentos) considera carência, limite de uso/período/escopo, excedente e gratuidade/grupo de gratuidade. A RPC fila_caixa_hoje reimplementa a seleção de regra em SQL, mas ignora todos esses campos — só considera modo/valor/percentual/prioridade.

Evidência:

`fila_caixa_hoje (SQL) seleciona de cb_convenio_regras sem considerar carencia_mensalidades, limite_qtd, excedente_modo, gratuito.`

Cenário para reproduzir: Paciente com contrato CB que ainda não cumpriu carência, ou já excedeu limite_qtd do período — a Agenda bloquearia/ajustaria o valor corretamente; a fila do Caixa mostra o valor da regra "crua", ignorando carência/limite.

Impacto técnico: Duas implementações da mesma regra de negócio divergindo em produção.

Impacto operacional: Operador do caixa pode confiar no valor sugerido pela fila sem conferir na Agenda.

Impacto financeiro/clínico: Cobrança a menor (ou gratuidade indevida) para pacientes fora de carência/limite.

Causa provável: Regra reimplementada em SQL para performance da fila, sem reusar a lógica TypeScript central.

Correção recomendada: Fazer fila_caixa_hoje chamar a mesma função de regra usada no restante do sistema (extrair para function SQL compartilhada), ou eliminar o cálculo independente e usar sempre o valor já gravado no agendamento.

Risco de regressão: Médio — mudança em RPC de leitura usada na tela principal do caixa; testar valores exibidos após a correção.

Testes necessários: Paciente fora de carência e paciente no limite de uso — confirmar que a fila do Caixa mostra o mesmo valor que a Agenda.

ALTA-12 — Vínculo agendamento-orçamento continua não-atômico (achado do relatório de 11/07 confirmado sem alteração)

ALTA

Categoria: Integridade referencial | **Aba/fluxo:** Vendas/Agenda — agendamento originado de orçamento

```
src/lib/agenda/criar-agendamento.functions.ts:293-312; caller app.agenda.tsx:2864-2866 — função/trecho: Passo 7 (vínculo com agendamento_orcamento_itens) roda depois do passo 6 (INSERT/UPDATE do agendamento), já commitado
```

Tabela/RPC/trigger: agendamento_orcamento_itens

Descrição: O agendamento já foi commitado antes do vínculo ser tentado. Falha no vínculo não reverte o agendamento — apenas retorna ok:true com vinculo_warning, traduzido em toast de aviso. Não há bloqueio, retry ou fila de reconciliação.

Evidência:

```
let vinculo_warning; if (...) { const {error: vErr} = await supabase.from("agendamento_orcamento_itens").insert(...); if (vErr) vinculo_warning = {...}; } return {ok:true, ..., vinculo_warning};
```

Cenário para reproduzir: Orçamento com itens selecionados para vincular; insert em agendamento_orcamento_itens falha (RLS, constraint, timeout); agendamento existe normalmente, mas o orçamento nunca marca os itens como consumidos.

Impacto técnico: Estado parcial permanente sem mecanismo de correção automática.

Impacto operacional: Relatórios de orçamento (itens pendentes x agendados) ficam incorretos; item do orçamento pode ser "esquecido" e nunca cobrado, ou re-selecionável indevidamente em outro agendamento.

Causa provável: Passo 6+7 não unificados numa RPC atômica (padrão já usado em salvar_agendamento_multi_imagem e pagar_repasse_medico, no mesmo módulo).

Correção recomendada: Mover os dois passos para uma RPC atômica única.

Risco de regressão: Médio — mudança estrutural na função central de criação; testar todos os fluxos que passam por ela.

Testes necessários: Simular falha no insert de vínculo e confirmar que o agendamento também não é criado (rollback completo).

ALTA-13 — Autoatendimento faz check-in do agendamento errado (primeiro do dia, sem filtrar status/proximidade)

ALTA

Categoria: Lógica de seleção incorreta | **Aba/fluxo:** Autoatendimento (kiosk)

```
src/routes/autoatendimento.tsx:233-274 — função/trecho: fazerCheckin() — busca sem
.neq("status","cancelado"), ordena por inicio ascendente, .limit(1)
```

Tabela/RPC/trigger: agendamentos

Descrição: Busca agendamentos do paciente no dia sem excluir cancelados, ordena por horário ascendente e pega só o primeiro — sempre o mais cedo do dia, mesmo cancelado, mesmo já finalizado, mesmo que o paciente tenha uma consulta posterior diferente hoje.

Evidência:

```
autoatendimento.tsx:241-249 — select sem .neq("status","cancelado"); .order("inicio").limit(1). :267-272
— só evita regressão se já em triagem/atendimento, não valida status.
```

Cenário para reproduzir: Paciente tem exame às 8h (já finalizado) e consulta às 14h; check-in no totem às 13h50 faz check-in do exame das 8h (o primeiro do dia), não da consulta das 14h.

Impacto técnico: Seleção por .limit(1) sem desambiguação por horário mais próximo/status válido.

Impacto operacional: Paciente confirma presença errada; tela de sucesso mostra horário/procedimento errado ao paciente; se o mais cedo do dia estiver cancelado, aceitaria check-in dele também.

Causa provável: Query simplificada sem considerar múltiplos agendamentos no mesmo dia.

Correção recomendada: Adicionar .neq("status","cancelado") e .in("fluxo_etapa",["aguardando_recepcao","recepcao"]); escolher o agendamento com inicio mais próximo do horário atual, não necessariamente o primeiro do dia.

Risco de regressão: Baixo — ajuste de query de leitura.

Testes necessários: Paciente com 2 agendamentos no mesmo dia (um cancelado) — confirmar que o check-in seleciona o correto.

ALTA-14 — Triagem sem idempotência — duplo clique gera senha duplicada; sem UNIQUE(agendamento_id) em triagens_enfermagem

ALTA

Categoria: Concorrência / duplicidade | **Aba/fluxo:** Triagem — Enfermagem

```
src/routes/_authenticated/app.triagem-enfermagem.tsx:175-199 (chamarPaciente), :201-282 (salvarEAvancar) — função/trecho: chamarPaciente() sem estado busy/disabled; salvarEAvancar() sem checar triagem existente para o agendamento
```

Tabela/RPC/trigger: senhas, triagens_enfermagem (sem UNIQUE(agendamento_id), confirmado em supabase/migrations/20260520194916_...sql:1-27)

Descrição: chamarPaciente() faz INSERT em senhas sem nenhum estado de "carregando"/disabled que impeça duplo clique (diferente de app.recepcao.tsx, que usa busy/disabled). salvarEAvancar() sempre insere em triagens_enfermagem sem checar se já existe triagem para aquele agendamento_id, e a tabela não tem UNIQUE(agendamento_id).

Evidência:

```
triagem-enfermagem.tsx:191-195 — insert("senhas") sem guard de estado. :258-259 — insert("triagens_enfermagem", rows) sem checagem de duplicidade prévia.
```

Cenário para reproduzir: Clique duplo em "Chamar" gera duas senhas para o mesmo paciente na fila; reabrir o card e salvar de novo cria dois registros de triagem para a mesma visita (o prontuário só pega o mais recente via order by created_at desc limit 1 — dados antigos ficam invisíveis mas não removidos).

Impacto técnico: Nenhuma constraint de banco impede múltiplas linhas de triagem por agendamento; nenhum guard de estado local no clique de "Chamar".

Impacto operacional: Fila de recepção com senhas fantasmas duplicadas; histórico de triagem com registros redundantes/conflitantes.

Causa provável: Tela implementada sem o mesmo padrão de disabled/busy usado em app.recepcao.tsx.

Correção recomendada: Adicionar busy/disabled no botão "Chamar" (mesmo padrão de app.recepcao.tsx); antes do insert de triagem, checar se já existe registro para aquele agendamento_id "hoje" e oferecer editar em vez de duplicar.

Risco de regressão: Baixo — ajuste de UI e checagem adicional antes de insert.

Testes necessários: Duplo clique em "Chamar" (deve gerar só uma senha); tentar salvar triagem duas vezes para o mesmo agendamento (deve editar, não duplicar).

Achados — Severidade Média

Duplicação de lógica entre v1/v2, lacunas de validação e dívida técnica relevante — não bloqueiam o uso, mas aumentam risco de regressão e inconsistência ao longo do tempo.

MED-01 — v1 e v2 reimplementam cancelamento/reagendamento de forma independente

MÉDIA

Categoria: Duplicação de lógica entre módulos | **Aba/fluxo:** Agenda clássica vs. Agenda V2

```
app.agenda.tsx (lógica inline) vs. status-agendamento.functions.ts / reagendar-agendamento.functions.ts — função/trecho: —
```

Tabela/RPC/trigger: agendamentos

Descrição: v1 não importa atualizarStatusAgendamento nem reagendarAgendamento — usa lógica própria inline. v2 usa exclusivamente as server functions. São 4 implementações distintas para as mesmas 2 operações de negócio, o que já causou a divergência dos achados CRIT-07/CRIT-08 (correção aplicada em um lado, não no outro).

Evidência:

```
grep por atualizarStatusAgendamento|reagendarAgendamento em app.agenda.tsx: nenhuma ocorrência.
```

Cenário para reproduzir: Bug corrigido em um lado permanece no outro — já aconteceu (RPC transacional de reagendamento aplicada só em v1).

Impacto técnico: Alto risco de regressão cruzada.

Impacto operacional: Correções precisam ser replicadas manualmente nos dois lados.

Causa provável: Migração incremental v1→v2 sob feature flag sem checar as duas versões ao corrigir bug (contraria a própria regra 10 de frontend.md do projeto).

Correção recomendada: Ao corrigir CRIT-07/CRIT-08, aplicar em ambas as implementações, ou migrar v2 para reusar a mesma RPC/trigger de v1.

Risco de regressão: —

Testes necessários: —

MED-02 — payload as never mascara checagem de tipos entre formulário e criarAgendamento

MÉDIA

Categoria: Qualidade de código / segurança de tipos | **Aba/fluxo:** Agenda clássica

```
src/routes/_authenticated/app.agenda.tsx:2841 — função/trecho: payload: payload as never
```

Tabela/RPC/trigger: —

Descrição: Contorna a checagem de tipos do TypeScript entre o objeto montado no formulário e o tipo esperado pela server function — silencia qualquer erro de tipo real em vez de sinalizá-lo em compilação.

Evidência:

```
payload: payload as never
```

Cenário para reproduzir: Mudança futura no tipo esperado não gera erro de build neste call-site.

Impacto técnico: Payload malformado pode chegar ao servidor sem aviso do compilador.

Impacto operacional: —

Causa provável: form state do componente não é fortemente tipado (usa as any em vários pontos).

Correção recomendada: Tipar explicitamente o payload como CriarAgendamentoInput["payload"] e remover o as never.

Risco de regressão: Baixo

Testes necessários: tsc --noEmit sem novos erros após a correção.

MED-03 — Falta validação "paciente já agendado no mesmo horário" e "data no passado"

MÉDIA

Categoria: Validação ausente | **Aba/fluxo:** Criação de agendamento (todos os caminhos)

```
app.agenda.tsx (submit), criar-agendamento.functions.ts (handler completo) — função/trecho: —
```

Tabela/RPC/trigger: agendamentos

Descrição: Nenhum dos três caminhos de criação checa se o mesmo paciente já tem outro agendamento sobreposto, nem bloqueia data/hora no passado.

Evidência:

```
Nenhuma comparação com new Date() bloqueando passado encontrada nos três arquivos.
```

Cenário para reproduzir: Paciente agendado duas vezes em horários conflitantes; agendamento retroativo criado por engano de digitação sem confirmação.

Impacto técnico: —

Impacto operacional: Paciente com compromissos conflitantes; lançamentos retroativos não intencionais.

Causa provável: Validação nunca implementada.

Correção recomendada: Adicionar checagem de sobreposição por paciente_id antes de salvar; exigir confirmação explícita para datas passadas.

Risco de regressão: Baixo

Testes necessários: Tentar criar 2º agendamento sobreposto para o mesmo paciente; tentar criar com data passada.

MED-04 — Geração de slots roda 100% no cliente, contradizendo a arquitetura documentada

MÉDIA

Categoria: Arquitetura / única fonte de verdade | **Aba/fluxo:** Disponibilidades — "Gerar agenda"

```
src/routes/_authenticated/app.disponibilidades.tsx:300-366 — função/trecho: gerarAgenda()
```

Tabela/RPC/trigger: agendamentos

Descrição: Calcula slotsPreview inteiramente em JS no navegador e faz delete+insert direto via client Supabase — sem passar por server function, contrariando docs/agenda/arquitetura.md ("Nada mais grava em agendamentos. Nada.").

Evidência:

```
app.disponibilidades.tsx:319-359 — delete + insert direto do browser.
```

Cenário para reproduzir: Mudança futura na regra de geração de slots precisa ser replicada corretamente só neste arquivo — não há ponto único de verdade.

Impacto técnico: Divergência entre documentação e código real; risco de lógica de geração divergir entre telas futuras.

Impacto operacional: Nenhuma auditoria centralizada de geração de agenda com contexto (quem gerou, com quais parâmetros).

Causa provável: Tela anterior à consolidação arquitetural documentada, nunca migrada.

Correção recomendada: Extrair para server function gerarAgendaMedico(...), mantendo a mesma lógica.

Risco de regressão: Médio

Testes necessários: Gerar agenda antes/depois da mudança e comparar resultado idêntico.

MED-05 — Timezone: geração de slots (browser) e validação de "agenda aberta" (server) usam calendários potencialmente diferentes

MÉDIA

Categoria: Timezone / consistência de datas | **Aba/fluxo:** Criação de agendamento + Disponibilidades

`src/lib/agenda/criar-agendamento.functions.ts:153-156 vs. app.disponibilidades.tsx:210-222,330-332 — função/trecho: —`

Tabela/RPC/trigger: —

Descrição: criarAgendamento calcula o dia com new Date(payload.inicio).getFullYear/Month/Date — métodos que usam o fuso do runtime (Cloudflare Workers = UTC). A geração de slots roda no navegador (fuso local, presumivelmente America/Sao_Paulo). date-utils.ts já documenta esse tipo de armadilha, mas criar-agendamento.functions.ts não usa os helpers existentes.

Evidência:

`date-utils.ts:1-10 documenta o padrão correto (hojeBR()); criar-agendamento.functions.ts usa Date bruto no servidor.`

Cenário para reproduzir: Horários de atendimento a partir de ~21h BRT (já é o dia seguinte em UTC) ficam sujeitos a descompasso entre o dia calculado pelo servidor e o dia assumido pelo browser.

Impacto técnico: Falso negativo "médico não tem agenda aberta nessa data" perto da virada, ou correspondência com slots do dia errado.

Impacto operacional: Risco maior em clínicas com expediente noturno (19h-22h). Não confirmado em runtime.

Causa provável: Cálculo de "dia" não padronizado com fuso explícito nos dois lados.

Correção recomendada: Padronizar o cálculo do dia usando America/Sao_Paulo explicitamente (Intl.DateTimeFormat, como hojeBR()) tanto na geração quanto na validação.

Risco de regressão: Baixo-médio

Testes necessários: Testar criação/validação de agendamento entre 21h e 23h59 BRT.

MED-06 — "BLOQUEIO" é tratado como slot livre pela mesma função que libera "DISPONÍVEL"

MÉDIA

Categoria: Nomenclatura invertida / risco latente | **Aba/fluxo:** Agenda clássica, criação, reagendamento

`app.agenda.tsx:120-123, criar-agendamento.functions.ts:106-109, reagendar-agendamento.functions.ts:32-35 — função/trecho: isSlotLivre(nome) — retorna true para "disponivel" OU "bloqueio"`

Tabela/RPC/trigger: agendamentos

Descrição: Um slot rotulado "BLOQUEIO" é considerado agendável pela mesma lógica que libera um "DISPONÍVEL" — inversão da semântica esperada (bloqueio deveria significar indisponível). Não há hoje nenhuma feature ativa que crie linhas "BLOQUEIO" (só um comentário vestigial), mas a função existe replicada em 3 arquivos.

Evidência:

`Código idêntico replicado nos 3 arquivos: isSlotLivre(nome) inclui "bloqueio" como liberado.`

Cenário para reproduzir: Se uma feature futura marcar horário como "BLOQUEIO" pensando em torná-lo indisponível, o sistema faria o oposto.

Impacto técnico: Inversão de regra esperada caso a nomenclatura seja reativada sem revisar esta função.

Impacto operacional: —

Causa provável: Nomenclatura legada de versão anterior, mantida sem feature correspondente ativa.

Correção recomendada: Redefinir a semântica antes de qualquer feature de bloqueio manual ser construída, ou remover "bloqueio" de isSlotLivre.

Risco de regressão: Baixo

Testes necessários: —

MED-07 — Não existe feature de bloqueio manual pontual de agenda (reunião, cirurgia, ausência)

MÉDIA

Categoria: Lacuna funcional | **Aba/fluxo:** Disponibilidades

```
não localizado em src/ nem supabase/migrations/ — função/trecho: —
```

Tabela/RPC/trigger: —

Descrição: Busca por bloqueio_agenda, agenda_bloqueio, excecao_agenda, folga, ferias, afastamento combinados com médico não retornou nenhuma tabela/rota/componente/RPC dedicada. A única forma prática hoje é não gerar slots "DISPONÍVEL" para o intervalo, ou apagar manualmente linhas de configuração semanal (afeta a regra recorrente inteira, não um dia específico).

Evidência:

```
—
```

Cenário para reproduzir: Médico precisa bloquear uma tarde para cirurgia — não há suporte de produto, recepção precisa manipular manualmente os slots do dia.

Impacto técnico: —

Impacto operacional: Alto risco de erro humano (esquecer de remover/gerar de volta), sem histórico de "por que este médico não tem agenda neste dia".

Causa provável: Funcionalidade nunca implementada.

Correção recomendada: Se o produto quer essa feature, desenhar tabela própria (medico_bloqueios_agenda com medico_id, inicio, fim, motivo, clinica_id + RLS) e considerá-la na validação de disponibilidade (junto com CRIT-04/CRIT-05).

Risco de regressão: —

Testes necessários: —

MED-08 — Sobreposição/contradição entre regras de disponibilidade não é impedida no cadastro

MÉDIA

Categoria: Validação ausente | **Aba/fluxo:** Disponibilidades — cadastro de regras semanais

```
supabase/migrations/20260516222526_...sql:1-14 (só CHECK hora_fim>hora_inicio);
app.disponibilidades.tsx:150-171 (adicionar()) — função/trecho: adicionar() insere sem checar conflito com regras existentes
```

Tabela/RPC/trigger: medico_disponibilidades

Descrição: Não há constraint impedindo duas linhas para o mesmo médico+dia da semana com horários e vigências sobrepostos. slotsPreview simplesmente agrega todas as regras que "batem" no dia/vigência, gerando possível duplicidade sem alertar o usuário.

Evidência:

```
Cadastrar "Seg 08h-12h vigência ano todo" e depois "Seg 09h-10h vigência só julho" gera duas regras conflitantes consideradas simultaneamente.
```

Cenário para reproduzir: Geração de slots incoerente (horários duplicados/parcialmente sobrepostos) dependendo da ordem/combinação cadastrada.

Impacto técnico: —

Impacto operacional: Agenda gerada incorretamente sem aviso ao operador.

Causa provável: Validação de sobreposição nunca implementada na UI nem no banco.

Correção recomendada: Validar sobreposição na UI ao adicionar/editar; considerar EXCLUDE constraint no banco por médico/agenda/dia da semana + faixa de horário + vigência.

Risco de regressão: Baixo

Testes necessários: Cadastrar regras conflitantes e confirmar aviso/bloqueio.

MED-09 — Feriados hardcoded, incompletos, client-side apenas, e duplicados em 2 arquivos

MÉDIA

Categoria: Duplicação de lógica / dado incompleto | **Aba/fluxo:** Disponibilidades

```
app.disponibilidades.tsx:26-43, src/components/enfermagem-horarios-parts.tsx:16-23 — função/trecho: FERIADOS_FIXOS
```

Tabela/RPC/trigger: —

Descrição: Set fixo com os mesmos 9 feriados nacionais de data fixa nos dois arquivos — não inclui feriados móveis (Carnaval, Sexta-feira Santa, Corpus Christi) nem estaduais/municipais, não configurável por clínica. Usado só na pré-visualização de geração; não há validação equivalente no servidor/banco impedindo agendar manualmente num feriado.

Evidência:

```
Lista idêntica duplicada nos dois arquivos.
```

Cenário para reproduzir: Clínica em estado/município com feriados diferentes, ou ano com Carnaval em dia "útil" do calendário fixo — geração inclui/exclui dias incorretamente sem aviso.

Impacto técnico: —

Impacto operacional: Agenda gerada com dias errados sem sinalização.

Causa provável: Lista hardcoded nunca migrada para configuração.

Correção recomendada: Mover para tabela feriados configurável (nacional + municipal) ou lib de cálculo de feriados móveis; eliminar duplicação entre os dois arquivos.

Risco de regressão: Baixo

Testes necessários: —

MED-10 — Reagendamento não sincroniza fila de recepção/triagem quando muda de dia

MÉDIA

Categoria: Consistência de fluxo | **Aba/fluxo:** Reagendamento

```
src/lib/agenda/reagendar-agendamento.functions.ts:20-22,182-195 — função/trecho: —
```

Tabela/RPC/trigger: agendamentos.fluxo_etapa

Descrição: reagendarAgendamento intencionalmente não altera fluxo_etapa (documentado no próprio arquivo) — coerente para reagendamento no mesmo dia, mas se o novo inicio cai em outro dia, o agendamento desaparece silenciosamente das listas de checkin/triagem (filtradas por dia) mesmo que o paciente já tivesse feito check-in.

Evidência:

```
Comentário do arquivo confirma que fluxo_etapa não é tocado.
```

Cenário para reproduzir: Paciente que já confirmou presença é reagendado (por engano ou decisão médica) para outro dia — recepção não recebe nenhum sinal de que ele "sumiu" da fila do dia atual.

Impacto técnico: —

Impacto operacional: Confusão operacional, paciente esperando sem estar mais na fila do dia.

Causa provável: Comportamento intencional para o caso comum (mesmo dia), não cobre o caso de mudança de dia.

Correção recomendada: Ao mover inicio para fora do dia atual em agendamento com fluxo_etapa diferente de aguardando_recepcao, emitir aviso para quem reagenda.

Risco de regressão: Baixo

Testes necessários: —

MED-11 — useRealtimeRefresh sem filtro de clínica gera refetch cross-tenant desnecessário

MÉDIA

Categoria: Performance / isolamento (sem vazamento de dado) | **Aba/fluxo:** Atendimento (médico/IA)

`src/hooks/use-realtime-refresh.ts:22-38, usado em app.atendimento-ia.$agendamentoId.tsx:187-191 — função/trecho: —`

Tabela/RPC/trigger: agendamentos, triagens_enfermagem, fin_lancamentos, agendamento_orcamento_itens

Descrição: A subscrição postgres_changes não usa filter por clinica_id (diferente de app.recepcao.tsx e app.triagem-enfermagem.tsx, que filtram corretamente). Toda alteração nessas tabelas de qualquer clínica do sistema dispara refetch nesta aba.

Evidência:

```
Comparar com app.recepcao.tsx:115-122 (filtra corretamente).
```

Cenário para reproduzir: Ambiente multi-tenant com muitas clínicas simultâneas gera carga/latência desnecessária.

Impacto técnico: Sem vazamento de dado (query final ainda filtra por id/RLS), mas desperdício de carga.

Impacto operacional: —

Causa provável: Hook não recebeu parâmetro de filtro nesta chamada específica.

Correção recomendada: Estender useRealtimeRefresh para aceitar filter opcional e usar clinica_id=eq..

Risco de regressão: Baixo

Testes necessários: —

MED-12 — Webhook WhatsApp não verifica cfg.ativo antes de processar

MÉDIA

Categoria: Regra de negócio ausente | **Aba/fluxo:** WhatsApp / Nina

`src/routes/api/public/whatsapp.$clinicaId.ts:47-51 — função/trecho: POST handler`

Tabela/RPC/trigger: whatsapp_configs.ativo

Descrição: O handler verifica app_secret/phone_number_id/access_token, mas nunca cfg.ativo.

Evidência:

```
if (!cfg.app_secret || !cfg.phone_number_id || !cfg.access_token) — sem checar cfg.ativo.
```

Cenário para reproduzir: Clínica desativa o WhatsApp (ativo=false) mas mantém credenciais preenchidas — mensagens continuam sendo gravadas e a Nina continua respondendo fora do horário.

Impacto técnico: —

Impacto operacional: Comportamento inconsistente com a intenção do toggle "ativo".

Causa provável: Flag ativo provavelmente usada só como indicador de UI.

Correção recomendada: Checar cfg.ativo no POST; se false, apenas persistir a mensagem sem acionar a Nina.

Risco de regressão: Baixo

Testes necessários: Desativar WhatsApp da clínica e confirmar que a Nina para de responder.

MED-13 — clinica_id duplicado e potencialmente inconsistente dentro de criarAgendamento, sem validação cruzada

MÉDIA

Categoria: Arquitetura / validação ausente | **Aba/fluxo:** Criação de agendamento

`src/lib/agenda/criar-agendamento.functions.ts:28-61,99-119,152-194 — função/trecho: inputValidator sem zod (pass-through)`

Tabela/RPC/trigger: agendamentos

Descrição: O objeto recebido tem dois campos clinica_id redundantes: data.clinica_id (usado nas checagens de conflito) e data.payload.clinica_id (usado no INSERT/UPDATE real). Nada garante que sejam iguais. O inputValidator também não usa zod — só tipagem TS, que não protege payload malicioso.

Evidência:

```
.eq("clinica_id", clinica_id) na checagem de slot livre; .insert(payloadNovo) usa payload.clinica_id, potencialmente diferente.
```

Cenário para reproduzir: Usuário membro de duas clínicas chama a função com clinica_id=A (checagem de conflito roda contra A) e payload.clinica_id=B (gravação em B) — checagem de horário roda contra a clínica errada.

Impacto técnico: Inconsistência entre clínica validada e clínica gravada.

Impacto operacional: Mitigado hoje pela RLS (que ainda impede gravar em clínica da qual o usuário não é membro), mas checagem de conflito de horário pode rodar contra a clínica errada quando o usuário é membro de ambas.

Causa provável: Refatoração que extraiu a lógica do submit clássico manteve os dois clinica_id sem unificação.

Correção recomendada: Validar data.clinica_id === data.payload.clinica_id no início do handler; adicionar .parse() via zod no inputValidator.

Risco de regressão: Baixo

Testes necessários: Chamar com os dois campos diferentes e confirmar rejeição após a correção.

MED-14 — Falta trigger garantindo que medico_id pertence à mesma clinica_id do agendamento

MÉDIA

Categoria: Integridade referencial | **Aba/fluxo:** Banco de dados

`FK simples em supabase/migrations/20260517052636_...sql:1-3 — função/trecho: —`

Tabela/RPC/trigger: agendamentos.medico_id → medicos.id

Descrição: medico_id tem FK simples para medicos(id), mas nada impede que um médico de uma clínica seja usado num agendamento de outra clínica (ambos IDs válidos individualmente).

Evidência:

```
ADD CONSTRAINT agendamentos_medico_id_fkey FOREIGN KEY (medico_id) REFERENCES medicos(id) ON DELETE SET NULL — sem checagem cruzada de clinica_id.
```

Cenário para reproduzir: Payload com clinica_id=B e medico_id de médico cadastrado na clínica A é aceito pelo INSERT.

Impacto técnico: Dado incoerente que pode confundir relatórios de repasse médico, ocupação de agenda e a própria contagem de conflito de horário.

Impacto operacional: Não é vazamento cross-tenant (RLS ainda protege acesso), mas é falha de integridade referencial explorável por erro de UI ou chamada direta.

Causa provável: Nunca implementado.

Correção recomendada: Trigger BEFORE INSERT OR UPDATE validando EXISTS (SELECT 1 FROM medicos WHERE id=NEW.medico_id AND clinica_id=NEW.clinica_id) — padrão semelhante ao já usado em fn_agendamento_valida_destino.

Risco de regressão: Médio

Testes necessários: Tentar criar agendamento com médico de outra clínica (deve ser bloqueado após a correção).

MED-15 — Trigger de auditoria removida de pacientes/fin_lancamentos desde 03/06; documentação interna desatualizada

MÉDIA

Categoria: Auditoria (fora do escopo estrito de Agendamento, mas descoberto na checagem de triggers) | **Aba/fluxo:** Banco de dados

```
criada em supabase/migrations/20260520160551_...sql:77-80; removida (com
pacientes/fin_lancamentos/procedimentos) em 20260603165959_...sql:2-5; recriada só para agendamentos em
20260709104608_...sql:1-3 — função/trecho: trg_audit_*
```

Tabela/RPC/trigger: pacientes, fin_lancamentos, procedimentos (sem trigger), agendamentos (com trigger)

Descrição: docs/contexto-do-projeto.md afirma que pacientes/fin_lancamentos/medicos/procedimentos/gr_impressoes/orcamento_itens/agendamentos têm trigger de auditoria. Hoje, por migração, só agendamentos tem — pacientes e fin_lancamentos perderam a trigger em 03/06 e não foram recriadas.

Evidência:

```
Migration 20260603165959 remove a trigger de 4 tabelas; 20260709104608 recria só para agendamentos.
```

Cenário para reproduzir: Alterações/exclusões em pacientes e fin_lancamentos não geram mais audit_log desde 03/06.

Impacto técnico: Documentação desatualizada.

Impacto operacional: Perda de trilha de auditoria em duas tabelas sensíveis (dado pessoal de saúde e dado financeiro).

Causa provável: Remoção provavelmente por performance/volume do audit_log; "reparo" cobriu só agendamentos.

Correção recomendada: Fora do escopo direto de Agendamento — sinalizar para o time avaliar recriar a trigger em pacientes/fin_lancamentos com throttling adequado.

Risco de regressão: —

Testes necessários: —

MED-16 — Estorno é multi-etapa client-side sem rollback compensatório, com falha silenciosa no caixa

MÉDIA

Categoria: Atomicidade / notificação de erro | **Aba/fluxo:** Financeiro — Estorno

```
src/lib/estornar-lancamento.ts:57-103,105-137 — função/trecho: estornarLancamentoReceita()
```

Tabela/RPC/trigger: fin_lancamentos, caixa_movimentos, agendamentos

Descrição: Executa 3 passos (cancelar lançamento, inserir sangria em caixa_movimentos dentro de try/catch que só faz console.warn sem notificar o usuário, reabrir status do agendamento) sem rollback compensatório se uma etapa intermediária falhar.

Evidência:

```
Passo 2 (sangria) em try/catch com apenas console.warn na linha 96-99 — usuário não é avisado.
```

Cenário para reproduzir: Passo 3 falha — lançamento já cancelado, sangria já lançada, mas agendamento continua com status antigo (pago) — inconsistência entre "financeiro estornado" e "agendamento ainda mostrado como pago".

Impacto técnico: Sem rollback compensatório.

Impacto operacional: Erro silencioso pode passar despercebido pelo operador.

Causa provável: Rotina não migrada para RPC atômica (padrão já usado em outras partes do módulo financeiro).

Correção recomendada: Mover para RPC atômica (mesmo padrão de fn_registrar_lancamento_e_caixa/pagar_repasse_medico); no mínimo, substituir console.warn por notificação visível.

Risco de regressão: Médio

Testes necessários: Simular falha em cada um dos 3 passos e confirmar comportamento consistente.

MED-17 — Totais "por forma de pagamento" e "resumo por tipo" no Caixa podem não excluir lançamentos cancelados de forma consistente

MÉDIA

Categoria: Relato financeiro (não confirmado em runtime) | **Aba/fluxo:** Caixa

```
src/routes/_authenticated/app.caixa.tsx:940-947,986-1017 vs. 920-938 — função/trecho: resumoTipos, entradasPorForma vs. saldoAtual
```

Tabela/RPC/trigger: caixa_movimentos

Descrição: saldoAtual filtra corretamente via lancsCancelados, mas resumoTipos e entradasPorForma iteram minhasMovs sem aplicar o mesmo filtro — duas bases de cálculo diferentes dentro do mesmo arquivo.

Evidência:

```
Comparação de código entre as três funções mostra filtros diferentes.
```

Cenário para reproduzir: Depende de a sangria de reversão ter sido lançada ou não — efeito exato não confirmado sem dados reais.

Impacto técnico: Inconsistência potencial de cálculo dentro do mesmo arquivo.

Impacto operacional: Relatório do caixa pode divergir do saldo real em cenários com estornos recentes.

Causa provável: Funções de resumo implementadas em momentos diferentes sem unificar a base de filtro.

Correção recomendada: Unificar a base de filtro (excluir cancelados) nas três funções.

Risco de regressão: Baixo

Testes necessários: Comparar resumoTipos/entradasPorForma com saldoAtual após um estorno.

MED-18 — Cancelamento não libera automaticamente o horário/slot

MÉDIA

Categoria: Comportamento a confirmar com a clínica | **Aba/fluxo:** Agenda clássica

```
app.agenda.tsx:3015-3036 (mudarStatus) vs. 2972-2983 (remove()) — função/trecho: —
```

Tabela/RPC/trigger: agendamentos

Descrição: mudarStatus(a,"cancelado") só altera status (e orcamento_id); mantém paciente_id/paciente_nome/procedimento intactos. Diferente de remove(), que zera esses campos e volta paciente_nome para "DISPONÍVEL". Um agendamento cancelado continua "ocupando" visualmente o slot até uma ação manual adicional.

Evidência:

```
Comparação direta entre as duas funções no mesmo arquivo.
```

Cenário para reproduzir: Recepção pode achar que o horário está livre pela ausência do paciente, mas o sistema não o oferece como DISPONÍVEL sem uma segunda ação manual.

Impacto técnico: Não é bug de dados — divergência de comportamento documentável.

Impacto operacional: Possível confusão operacional (não é falha de segurança/consistência).

Causa provável: Possivelmente intencional (preserva histórico/rastreabilidade).

Correção recomendada: Possível regra de negócio — validar com a equipe da clínica se cancelamento deve liberar automaticamente o horário.

Risco de regressão: —

Testes necessários: —

Achados — Severidade Baixa

BAIXA-01 — Import não utilizado (useMemo) no Wizard V2

BAIXA

Categoria: Código morto | **Aba/fluxo:** Agenda V2

```
src/components/agenda-v2/novo-agendamento-wizard.tsx:1 — função/trecho: import { useEffect, useMemo, useState }
```

Tabela/RPC/trigger: —

Descrição: useMemo não é referenciado em nenhum outro ponto do arquivo.

Evidência:

```
import { useEffect, useMemo, useState } from "react";
```

Cenário para reproduzir: —

Impacto técnico: Nenhum — cosmético.

Impacto operacional: —

Causa provável: Import esquecido após refatoração.

Correção recomendada: Remover useMemo do import.

Risco de regressão: Nenhum

Testes necessários: Lint sem novos avisos.

BAIXA-02 — Mutações de agendamentos dependem só de RLS, sem filtro de clínica explícito no client (defesa em profundidade ausente)

BAIXA

Categoria: Isolamento multi-clínica (mitigado hoje) | **Aba/fluxo:** Agenda clássica

```
app.agenda.tsx:1957-1959,2267,2749,3036 — função/trecho: update/delete sem .eq("clinica_id",...)
```

Tabela/RPC/trigger: agendamentos

Descrição: Várias mutações filtram só por id/lista de ids, sem repetir .eq("clinica_id",...). Seguro hoje porque a RLS está corretamente configurada, mas sem segunda camada de proteção caso a policy seja alterada incorretamente no futuro.

Evidência:

```
Comparar com checagem de cota de convênio (linha 438-443), que faz o filtro explícito corretamente.
```

Cenário para reproduzir: Risco latente, não vazamento hoje.

Impacto técnico: Nenhum hoje.

Impacto operacional: Nenhum hoje; risco futuro caso RLS seja reescrita com erro.

Causa provável: Padrão inconsistente entre partes do mesmo arquivo.

Correção recomendada: Adicionar .eq("clinica_id", clinicaAtual.clinica_id) nessas mutações como defesa em profundidade.

Risco de regressão: Nenhum

Testes necessários: —

BAIXA-03 — Duração de consulta é única por médico, não por procedimento

BAIXA

Categoria: Limitação de modelo de dados | **Aba/fluxo:** Disponibilidades

`app.disponibilidades.tsx:47,228,266-268 — função/trecho: —`

Tabela/RPC/trigger: medicos.duracao_consulta_min, medico_disponibilidades.intervalo_min

Descrição: Não há duração diferenciada por procedimento/serviço específico dentro da mesma regra de disponibilidade — confirma como não implementado o item do escopo original.

Evidência:

```
—
```

Cenário para reproduzir: —

Impacto técnico: —

Impacto operacional: Limitação de produto, não bug.

Causa provável: Modelo de dados nunca previu essa granularidade.

Correção recomendada: Avaliar com o produto se é um requisito real antes de desenhar a mudança de modelo.

Risco de regressão: —

Testes necessários: —

BAIXA-04 — Capacidade simultânea modelada como "limite por dia", não "N vagas no mesmo horário" — incompatível com o índice único de slot vazio

BAIXA

Categoria: Limitação de modelo de dados | **Aba/fluxo:** Disponibilidades

`app.disponibilidades.tsx:255-263,45-46 — função/trecho: limite_pacientes`

Tabela/RPC/trigger: agendamentos (uq_agend_slot_vazio)

Descrição: limite_pacientes limita quantos slots são gerados no dia, mas cada slot continua sendo 1 horário = 1 vaga. Não há modelo de "N pacientes no mesmo inicio" — para atingir isso hoje seria preciso gerar N linhas idênticas de horário, o que colidiria com o índice único uq_agend_slot_vazio.

Evidência:

```
Índice único por (clinica_id, medico_id, agenda_id, inicio) impede múltiplas linhas DISPONÍVEL idênticas.
```

Cenário para reproduzir: Exames em lote (vários pacientes no mesmo horário) não têm suporte claro no modelo atual.

Impacto técnico: —

Impacto operacional: Contradiz o requisito de "capacidade de atendimento simultâneo" do escopo original, a menos que se use agenda_id/enfermagem_recurso_id diferentes para cada vaga — não confirmado se é essa a solução adotada.

Causa provável: Modelo de dados não previu esse caso de uso.

Correção recomendada: Validar com a equipe da clínica como exames em lote são tratados hoje, e desenhar suporte explícito se necessário.

Risco de regressão: —

Testes necessários: —

Mapa de banco de dados — Agendamento

Tabelas

Tabela	Papel	Observação
agendamentos	Núcleo do módulo (slots + agendamentos reais)	Sem UNIQUE cobrindo overlap por médico (CRIT-02/CRIT-05); sem CHE
agendamentos_legacy_import	Staging de importação do sistema antigo	UNIQUE(clinica_id, codigo_agenda)
agendamento_orcamento_itens	Vínculo agendamento ↔ item de orçamento	UNIQUE(agendamento_id, orcamento_item_id); vínculo non-atômico (ALT
medico_agendas	Agendas nomeadas por médico	UNIQUE(medico_id, lower(nome))
medico_agenda_procedimentos	Vínculo agenda ↔ procedimento permitido	UNIQUE(agenda_id, procedimento_id)
medico_disponibilidades	Grade semanal de horários do médico	CHECK(hora_fim>hora_inicio); sem constraint anti-sobreposição (MED-08
enfermagem_recursos / _disponibilidades	Salas/equipamentos agendáveis	Bypass de validação de slot (ALTA-03)
gr_impressoes	Controle de impressão de guia por agendamento	agendamento_id sem FK declarada
senhas	Fila de atendimento	UNIQUE + advisory lock — padrão de referência (ver BAIXA/positivo)
anamnese_respostas	Respostas de anamnese vinculadas a agendamento	ON DELETE CASCADE — risco de perda de dado clínico (ALTA-09)
whatsapp_configs	Credenciais/config do WhatsApp por clínica	Guarda access_token/app_secret em texto puro
whatsapp_mensagens	Histórico de mensagens	UNIQUE(wa_message_id) — não usado como trava de idempotência (AL
fin_lancamentos / caixa_movimentos	Financeiro vinculado a agendamento_id	Sem sync automático com status do agendamento (CRIT-08)
prontuarios / fin_atendimentos	Dado clínico/financeiro do atendimento	Sem agendamento_id (ALTA-04)
triagens_enfermagem	Registro de triagem	Sem UNIQUE(agendamento_id) (ALTA-14)

Views

Nenhuma view dedicada a Agendamento foi encontrada no schema.

Triggers em agendamentos

Trigger	Evento	Observação
agendamentos_touch	BEFORE UPDATE	Padrão (updated_at)
trg_audit_agendamentos	AFTER INSERT/UPDATE/DELETE	Removida e recriada; outras tabelas críticas ficaram sem (MED-15)
uc_agendamentos_pac	BEFORE INSERT/UPDATE OF paciente_upper	Uppercase do nome; consistente com modelo de slot
trg_agendamento_valida_destino	BEFORE INSERT	Valida compatibilidade agendamento↔orçamento quando aplicável;

Funções/RPCs relacionadas

Função	Security	Valida is_member?	Observação
is_member / can_manage_clinica / has_role	DEFINER	(é a própria checagem)	Base de toda a RLS do módulo — corretas
fn_audit_trigger	DEFINER	n/a	Grava audit_log
consulta_publica / salvar_anamnese_publica	DEFINER	n/a (por token)	Acesso por token público — por design (portal do paciente)
minhas_consultas	DEFINER	usa auth.uid() do próprio usuário	Escopo correto (self)
fn_agendamento_valida_destino	DEFINER	n/a (trigger)	—
salvar_agendamento_multi_imagem	INVOKER	herda RLS do chamador	Correto — RLS de agendamentos se aplica
emitir_senha / chamar_proxima_senha	DEFINER	sim, explícito	Padrão de referência (advisory lock)
reagendar_atendimento (v1)	—	—	Transacional, FOR UPDATE — correta
finalizar_pagamento_agrupado	—	—	Atômica em si; não unificada com criação do lançamento principal (ALT

pagar_repasse_medico	—	—	Atômica, com trava otimista
fila_caixa_hoje	—	—	Reimplementa regra CB divergente (ALTA-11); sem filtro de cancelado
checkin_agendamento	DEFINER	não (é por token)	Sem validar status/data (CRIT-10)
buscar_paciente_contato	DEFINER	NÃO valida	CRÍTICO — GRANT a anon (CRIT-11)
paciente_cartao_inadimplente	DEFINER	NÃO valida	ALTO — acessível por authenticated de qualquer clínica (ALTA-08)

Síntese — riscos por categoria de impacto

Problemas críticos que podem causar agendamento duplicado

- CRIT-01 — "Novo agendamento" não consome slot existente (Agenda clássica)
- CRIT-02 — INSERT puro sem trava de concorrência nem constraint no banco
- CRIT-03 — Agenda Express sem trava de concorrência nenhuma
- ALTA-03 — Recursos de enfermagem pulam a trava de concorrência mesmo no ramo UPDATE

Problemas críticos que podem causar sobreposição de horário

- CRIT-02, CRIT-03, CRIT-05 (RLS não impede overlap no banco)
- CRIT-04 (validação de disponibilidade decidida pelo cliente, pode ser burlada)
- ALTA-03 (recursos de enfermagem)

Problemas que podem causar perda ou corrupção de dados

- ALTA-09 — CASCADE apaga anamnese_respostas ao excluir agendamento, sem auditoria
- CRIT-07 — slot "fantasma" travado permanentemente se reagendamento V2 falhar no meio
- ALTA-07 — sem CHECK fim>inicio, permite intervalo inválido no banco

Problemas que podem afetar pagamentos ou gerar prejuízo financeiro

- CRIT-08 — cancelar/faltar agendamento pago não exige estorno
- ALTA-01 — pagamento agrupado pode marcar atendimentos como pagos sem lançamento real
- ALTA-10 — checagem "já pago" inconsistente bloqueia recobrança pós-estorno
- ALTA-11 — fila do caixa ignora carência/limite do cartão de benefícios
- ALTA-12 — vínculo com orçamento non-atômico
- MED-16 — estorno sem rollback compensatório

Problemas que podem causar vazamento de dados entre clínicas

- CRIT-11 — buscar_paciente_contato exposta a chamada anônima (CRÍTICO — PII de qualquer clínica)
- ALTA-08 — paciente_cartao_inadimplente acessível por qualquer usuário autenticado de qualquer clínica
- MED-13, MED-14 — inconsistências de clinica_id sem validação cruzada (risco latente, mitigado por RLS hoje)

Pontos que não puderam ser validados nesta auditoria

• Comportamento real sob concorrência (duas requisições simultâneas) — os cenários descritos são inferidos logicamente do código (ausência de guard/lock/constraint), não observados em execução real, pois não há acesso a app rodando nem banco ao vivo.

• Se as políticas RLS descritas nas migrations são exatamente as ativas em produção hoje — só foi possível ler o histórico de migrations no repositório.

• Se os GRANT/REVOKE de buscar_paciente_contato e paciente_cartao_inadimplente (CRIT-11, ALTA-08) refletem o estado real do banco em produção.

• Volume real de uso de cada caminho de criação (clique em slot vs. "Novo agendamento" vs. Agenda Express) — afeta a frequência esperada dos achados críticos de concorrência, não a validade deles.

• Se a Meta Cloud API de fato reenvia webhooks duplicados neste ambiente específico (ALTA-06) — comportamento documentado como padrão da plataforma, não observado em log de produção.

• Se existe alguma trava ou trigger adicional fora do diretório supabase/migrations/ (ex. criada manualmente no painel do Supabase) que mitigue parcialmente algum achado de concorrência.

• Efeito real do achado MED-17 (totais do caixa) — depende de dados de produção com estornos recentes para observar divergência.

• Se EstornoAtendimentosPanel.tsx e conversao-orcamento-dialog.tsx (que referenciam um vocabulário de status próprio) estão de fato mortos/não utilizados ou acessíveis por alguma rota não capturada nesta auditoria.

• Comportamento de pagamentoStatus/agendamentoStatusPagamento (src/lib/pagamento-status.ts) — usado como caixa-preta em alguns achados, não auditado em profundidade.

• Corpo completo de ~30 migrations SECURITY DEFINER da leva de conversão de orçamento (2026-07-04/05) e das ~12 tabelas atend_* (fila/bot de WhatsApp) — mapeadas por nome, não lidas linha a linha (fora do foco direto de confirmação/lembrete de agendamento).

• Testes de UI/responsividade/acessibilidade em dispositivos reais (mobile, tablet, totem) — esta auditoria é de código estático; não há app rodando neste ambiente para validação visual.

• Performance real sob carga (queries N+1, tempo de resposta) — não foi possível medir sem ambiente de execução.

Ordem recomendada de correção e plano de fases

A ordem prioriza: (1) segurança/vazamento de dados — mais barato de corrigir e mais grave se explorado; (2) integridade de agendamento (duplicidade/sobreposição) — afeta a operação central todos os dias; (3) integridade financeira; (4) consistência de fluxo operacional; (5) dívida técnica e lacunas menores.

Fase 0 — Contenção imediata (horas, não dias)

- CRIT-11: REVOKE de buscar_paciente_contato de anon/PUBLIC — mudança de permissão isolada, baixíssimo risco de regressão, fecha o vazamento de PII mais grave.
- ALTA-08: adicionar checagem de is_member em paciente_cartao_inadimplente — mesma classe de correção, baixo risco.

Fase 1 — Concorrência e duplicidade de agendamento (1-2 semanas)

- CRIT-01: corrigir openNew()/atalho N/deep-link WhatsApp para consumir slot existente quando aplicável.
- CRIT-02: estender trava otimista ao ramo de INSERT puro; avaliar constraint de banco complementar.
- CRIT-04 + CRIT-05: servidor deve recalcular a necessidade de validação de disponibilidade, nunca confiar em flag do cliente; considerar mover a regra para trigger/RPC único caminho de escrita.
- ALTA-03: aplicar a mesma trava otimista para recursos de enfermagem.
- CRIT-03: migrar Agenda Express para reusar criarAgendamento (maior esforço desta fase — planejar à parte).

Fase 2 — Status, fluxo e check-in (1-2 semanas, em paralelo com Fase 1 se houver equipe)

- CRIT-06: decidir entre ALTER TYPE ou migrar V2 para fluxo_etapa; corrigir o check-in da V2.
- CRIT-07: implementar RPC transacional de reagendamento para a V2.
- CRIT-08: adicionar guard de pagamento em mudarStatus()/atualizarStatusAgendamento.
- CRIT-09: adicionar filtro de status nas filas de triagem/atendimento; revisar auto-promoção de fluxo_etapa.
- CRIT-10: adicionar validação de status/data na RPC checkin_agendamento.
- ALTA-02, ALTA-05, ALTA-13, ALTA-14: guards de estado e idempotência nas telas de status/check-in/triagem/autoatendimento.

Fase 3 — Integridade financeira (1-2 semanas)

- ALTA-01: unificar RPC de pagamento agrupado com a criação do lançamento principal.
- ALTA-10: centralizar checagem "já pago" numa função única.
- ALTA-11: unificar regra de cartão de benefícios entre Agenda e fila do Caixa.
- ALTA-12: unificar criação de agendamento + vínculo de orçamento numa RPC atômica.
- MED-16: mover estorno para RPC atômica com notificação visível de erro.

Fase 4 — Integridade de banco e dívida técnica (contínuo)

- ALTA-07 (CHECK fim>inicio), ALTA-09 (CASCADE de anamnese), MED-14 (médico×clínica), MED-08 (sobreposição de disponibilidade), MED-09 (feriados), MED-04/MED-05 (arquitetura de geração de slots e timezone).
- MED-01: consolidar v1/v2 numa única implementação de cancelamento/reagendamento assim que a V2 estiver estável o suficiente para promoção.
- Achados de severidade baixa conforme capacidade da equipe.

Baterias de teste recomendadas

O projeto não tem framework de testes automatizados configurado via npm (confirmado em package.json; existe ao menos um arquivo bun:test isolado — src/lib/agenda/aviso-limite-pendentes.test.ts — não integrado a nenhum script de CI). As baterias abaixo são recomendações de escopo para validação manual ou futura automação, não testes já existentes.

Testes funcionais

- Criar agendamento pelos 3 caminhos (slot, "Novo agendamento"/N, Agenda Express) e confirmar resultado consistente.
- Reagendar em v1 e v2; cancelar com e sem pagamento; marcar falta; concluir manualmente.
- Fluxo completo: disponibilidade → slot → criação → confirmação → check-in → triagem → atendimento → pagamento → finalização.
- Pagamento avulso, agrupado e com gratuidade.

Testes de validação

- Paciente/médico/serviço/data/horário obrigatórios ausentes.
- Data no passado, horário fora do expediente do médico, horário bloqueado.
- Paciente com cadastro incompleto nos 3 caminhos de criação (deve bloquear nos 2 corretos, hoje não bloqueia na Express).
- Duplo clique/envio duplicado em: criar agendamento, confirmar check-in, chamar senha na triagem, salvar triagem.

Testes de integração

- Cancelamento propaga (ou não) para recepção/triagem/atendimento/caixa.
- Reagendamento reflete corretamente em check-in, pagamento vinculado e notificações (quando existirem).
- Estorno reabre corretamente o agendamento e permite recobrança pelo Caixa.
- Webhook do WhatsApp: mensagem recebida gera resposta correta da Nina, sem duplicar.

Testes de concorrência

- Duas requisições simultâneas de criação para o mesmo médico/horário (pelos 3 caminhos) — deve falhar exatamente uma.
- Dois usuários reagendando o mesmo destino simultaneamente (v1 e v2).
- Dois usuários confirmando check-in do mesmo agendamento simultaneamente.
- Webhook do WhatsApp reentregue duas vezes para a mesma mensagem — deve responder só uma vez.
- Dois operadores atribuindo o mesmo recurso de enfermagem/horário simultaneamente.

Testes de permissões

- Usuário sem módulo "agenda:write" tentando criar/editar/excluir agendamento.
- Usuário de uma clínica tentando acessar/alterar agendamento de outra clínica (via API direta, não só pela UI).
- Chamada anônima (sem login) às RPCs buscar_paciente_contato e paciente_cartao_inadimplente após a correção — deve ser rejeitada.
- Papel médico tentando concluir/cancelar atendimento fora das permissões esperadas.

Testes de regressão

- Após cada correção de Fase 1-3, reexecutar o fluxo completo (disponibilidade → finalização) nas 3 combinações de agenda (v1, v2, Express).
- Confirmar que os achados do relatório de 11/07/2026 (já corrigidos parcialmente) continuam corrigidos após qualquer mudança na lógica de criação/reagendamento.
- Confirmar que a correção de um achado em v1 foi replicada em v2 quando aplicável (e vice-versa), dado o padrão de divergência já observado (MED-01).

Este relatório é somente de leitura/análise — nenhuma alteração de código, migration ou dado foi aplicada durante esta auditoria. Achados que exigem mudança de schema (CRIT-06, CRIT-11, ALTA-07/08/09, MED-14, entre outros) requerem nova migration, apresentada para revisão explícita antes de qualquer supabase db push, conforme as regras do projeto.